American International University-Bangladesh (AIUB)
Faculty of Science and Technology
Department of Computer Science

Web Technologies

CSE 4101 / [Course Code]

PROJECT REPORT

Project Title:

Online Quiz & Exam Platform

Semester: Spring 2025–2026
Submission Date: 19.05.26
Section: T

Submitted To:
MIR MD. KAWSUR
lecturer, Faculty
DEPARTMENT OF COMPUTER SCIENCE
American International University-Bangladesh

Group Information

Group Number: 05

Project Title: Online Quiz & Exam Platform

Student Information:

Full Name: Md. Murad Hasan

Student ID:23-55139-3

Program: Computer science and engineering

Email: 23-55559-3@student.aiub.edu

Signature: 

Declaration: We hereby declare that this project report is our original work. Any external resources or references used have been properly cited. We have not engaged in any form of academic dishonesty.

Marks Sheet

To be filled by the instructor only. Students must NOT write in this section.

Group No.		**Section**	
Member 1 ID			

CO3: Illustrate Multi-Tier Web Application for Targeted Society					
Feature Implementation (0-3)			**Feature Implementation (0-3)**		**Completeness (0-3)**
Basic Web Security	**UI (HTML/CSS)**	**Feature Completeness**	**DB**	**Auth (Session/cookie)**	**MVC**
CO4: Coordinate with Group Members to Develop Multi-Tier Application					
Technical Knowledge (0-3)			**Collaborative Teamwork (0-3)**		**Communication/Promptness (0-3)**
JS validation	**PHP Validation**	**Ajax/JSON**	**Git Contribution**		**VIVA**
Expected Outcome (0-2)		**CO3 Total**	**CO4 Total**		**Project Total**

Evaluated By: _____ **Date:** _____

Course Teacher Signature: _____

Table of Contents

1. Group Information 2
2. Marks Sheet 3
3. Table of Contents 4
4. Chapter 1: Project Overview 9
   4.1 Project Description 9
   4.1 Technologies Used 10
   4.3 System Architecture / MVC Diagram 11
   4.4 Database Schema 12
5. Chapter 2: Git Contribution & Task Allocation 13
   5.1 GitHub Repository Link 13
   5.2 Git Contribution Screenshot 14
6. Chapter 3: Features, Source Code & Screenshots 16
7. Chapter 5: Conclusion & Future Work 18

Chapter 1: Project Overview

1.1 Project Description

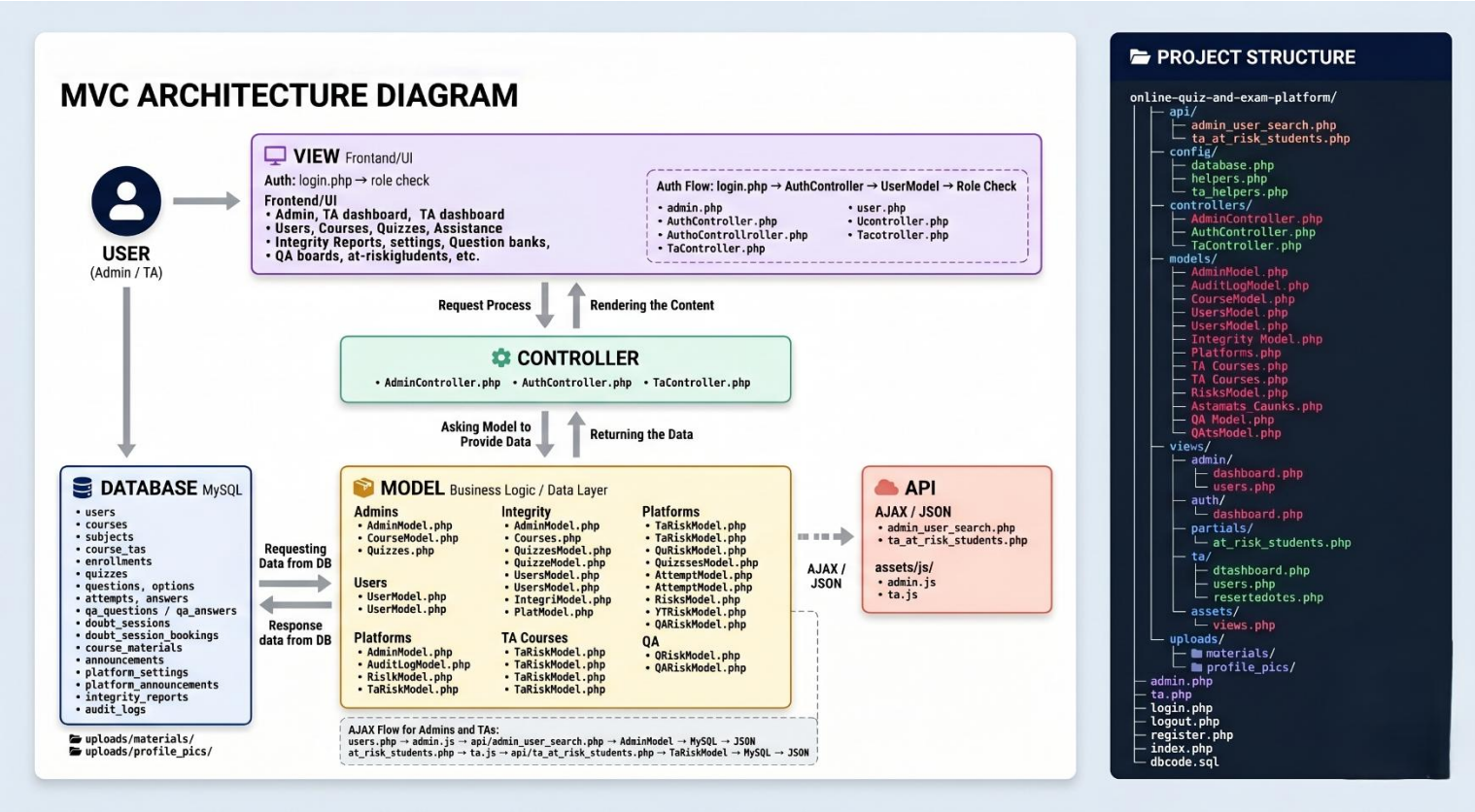
The Student Role of the Online Quiz & Exam Platform was designed to help students manage their learning activities in an easy and organized way. Students can create accounts, log in, enroll in courses, access learning materials, participate in quizzes, and view their results and performance history.

The system also allows students to interact through the Q&A board, read announcements, and book doubt sessions for additional academic support. A personal dashboard was created so students can quickly view enrolled courses, upcoming quizzes, quiz history, and overall progress in one place.

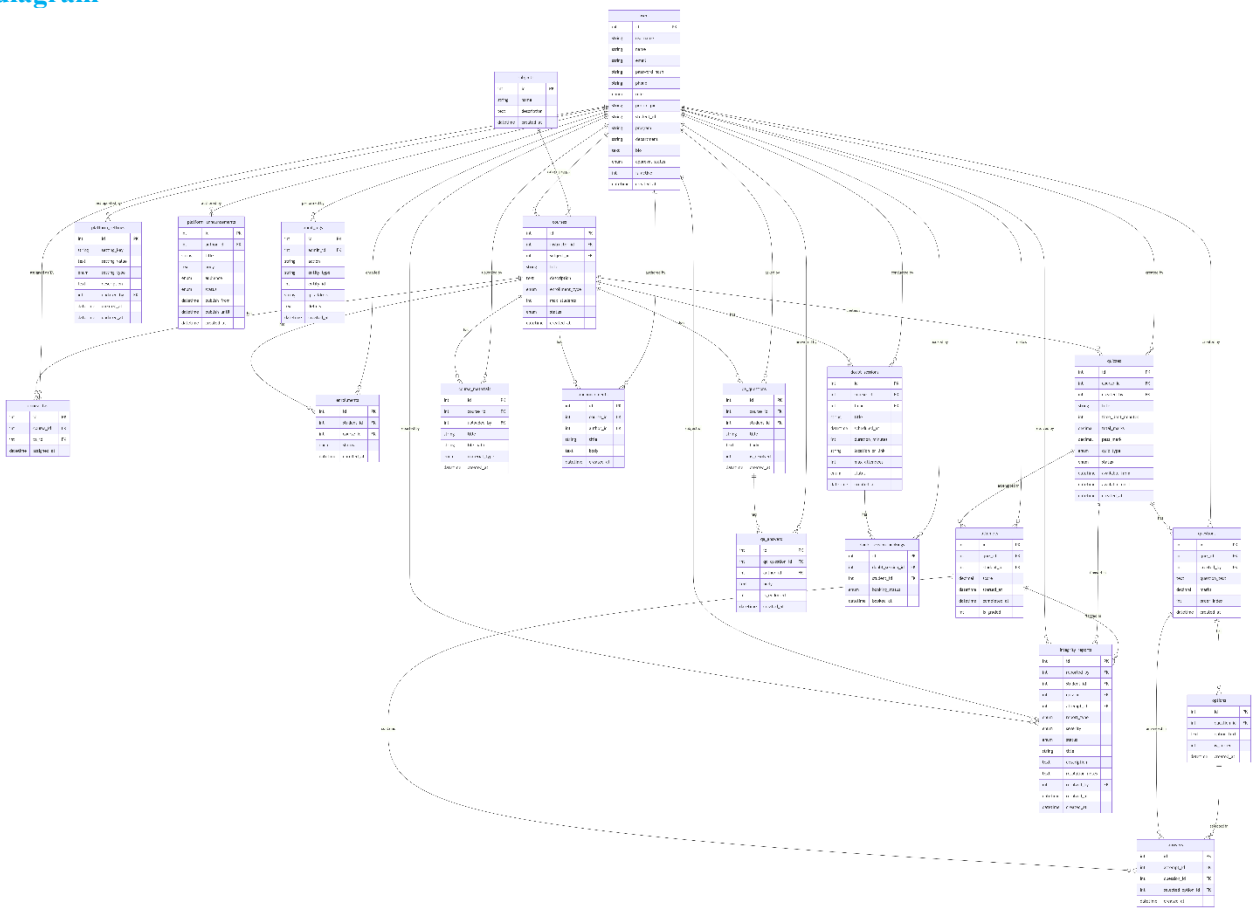
1.2 Technologies Used

Layer	Technology / Tool	Purpose
Frontend	HTML5, CSS3	Creating web pages and form structures, styling the Admin dashboard, TA dashboard, forms, tables, sidebar, and overall layout
Frontend	JavaScript, XMLHttpRequest, JSON	Client-side validation, confirmation messages for Admin and TA actions, and fetching dynamic data through AJAX without reloading the page
Backend	PHP	Server-side logic, routing, form handling, validation, session handling, and Admin/TA feature processing
Database	MySQL	Storing users, courses, quizzes, enrollments, attempts, Q&A, announcements, materials, doubt sessions, and admin-related records
Auth	PHP Sessions	Managing Admin and Teaching Assistant login, logout, active sessions, and role-based access control
Version Control	Git / GitHub	Group collaboration, version management, branching, merging, and contribution tracking
Server	XAMPP	Running the PHP project on localhost for development

1.3 System Architecture / MVC Diagram



ER diagram



1.4 Database Schema

Table Name	Field Name	Data Type	Description / Constraint
users	id	INT UNSIGNED	Primary Key, Auto Increment
	username	VARCHAR(50)	Unique username (nullable)
	name	VARCHAR(100)	Full name of the user (NOT NULL)
	email	VARCHAR(150)	Unique email address (NOT NULL)
	password_hash	VARCHAR(255)	Bcrypt hashed password (NOT NULL)
	phone	VARCHAR(30)	Contact phone number (nullable)
	role	ENUM	Values: student, instructor, ta, admin (NOT NULL)
	profile_pic	VARCHAR(255)	Profile picture file path (nullable)
	student_id	VARCHAR(50)	Unique student ID (nullable)
	program	VARCHAR(100)	Academic program (nullable)

Table Name	Field Name	Data Type	Description / Constraint
	department	VARCHAR(100)	Department name (nullable)
	bio	TEXT	Short biography (nullable)
	approval_status	ENUM	Values: pending, approved, rejected (Default: approved)
	is_active	TINYINT(1)	1 = Active, 0 = Inactive (Default: 1)
	created_at	TIMESTAMP	Account creation timestamp
	updated_at	TIMESTAMP	Last update timestamp (auto-updated)
subjects	id	INT UNSIGNED	Primary Key, Auto Increment
	name	VARCHAR(100)	Unique subject name (NOT NULL)
	description	TEXT	Subject description (nullable)
	created_at	TIMESTAMP	Creation timestamp
courses	id	INT UNSIGNED	Primary Key, Auto Increment
	instructor_id	INT UNSIGNED	FK → users.id (NOT NULL)
	subject_id	INT UNSIGNED	FK → subjects.id (NOT NULL)
	title	VARCHAR(150)	Course title (NOT NULL)
	description	TEXT	Course description (nullable)
	enrollment_type	ENUM	Values: open, approval (Default: open)
	max_students	INT UNSIGNED	Maximum enrolled students (Default: 50)
	status	ENUM	Values: draft, active, archived (Default: draft)
	created_at	TIMESTAMP	Creation timestamp
course_tas	id	INT UNSIGNED	Primary Key, Auto Increment
	course_id	INT UNSIGNED	FK → courses.id (NOT NULL)
	ta_id	INT UNSIGNED	FK → users.id (NOT NULL)
	assigned_at	TIMESTAMP	Assignment timestamp
enrollments	id	INT UNSIGNED	Primary Key, Auto Increment
	student_id	INT UNSIGNED	FK → users.id (NOT NULL)
	course_id	INT UNSIGNED	FK → courses.id (NOT NULL)
	status	ENUM	Values: pending, active, dropped (Default: pending)
	enrolled_at	TIMESTAMP	Enrollment timestamp
quizzes	id	INT UNSIGNED	Primary Key, Auto Increment

Table Name	Field Name	Data Type	Description / Constraint
	course_id	INT UNSIGNED	FK → courses.id (NOT NULL)
	created_by	INT UNSIGNED	FK → users.id (NOT NULL)
	title	VARCHAR(150)	Quiz title (NOT NULL)
	description	TEXT	Quiz description (nullable)
	time_limit_minutes	INT UNSIGNED	Time limit in minutes (Default: 30)
	total_marks	DECIMAL(8,2)	Total marks for the quiz (Default: 0.00)
	pass_mark	DECIMAL(8,2)	Minimum passing marks (Default: 0.00)
	quiz_type	ENUM	Values: graded, practice (Default: graded)
	status	ENUM	Values: draft, published (Default: draft)
	available_from	DATETIME	Quiz availability start (nullable)
	available_until	DATETIME	Quiz availability end (nullable)
	created_at	TIMESTAMP	Creation timestamp
questions	id	INT UNSIGNED	Primary Key, Auto Increment
	quiz_id	INT UNSIGNED	FK → quizzes.id (NOT NULL)
	question_text	TEXT	Question content (NOT NULL)
	marks	DECIMAL(8,2)	Marks for this question (Default: 1.00)
	order_index	INT UNSIGNED	Display order within quiz (Default: 1)
	created_by	INT UNSIGNED	FK → users.id (NOT NULL)
	created_at	TIMESTAMP	Creation timestamp
options	id	INT UNSIGNED	Primary Key, Auto Increment
	question_id	INT UNSIGNED	FK → questions.id (NOT NULL)
	option_text	TEXT	Answer option content (NOT NULL)
	is_correct	TINYINT(1)	1 = Correct answer, 0 = Wrong (Default: 0)
	created_at	TIMESTAMP	Creation timestamp
attempts	id	INT UNSIGNED	Primary Key, Auto Increment
	quiz_id	INT UNSIGNED	FK → quizzes.id (NOT NULL)
	student_id	INT UNSIGNED	FK → users.id (NOT NULL)
	score	DECIMAL(8,2)	Student's obtained score (Default: 0.00)
	started_at	DATETIME	Quiz attempt start time
	completed_at	DATETIME	Quiz attempt completion time (nullable)

Table Name	Field Name	Data Type	Description / Constraint
	is_graded	TINYINT(1)	1 = Graded, 0 = Pending (Default: 0)
answers	id	INT UNSIGNED	Primary Key, Auto Increment
	attempt_id	INT UNSIGNED	FK → attempts.id (NOT NULL)
	question_id	INT UNSIGNED	FK → questions.id (NOT NULL)
	selected_option_id	INT UNSIGNED	FK → options.id (nullable, SET NULL on delete)
	created_at	TIMESTAMP	Creation timestamp
course_materials	id	INT UNSIGNED	Primary Key, Auto Increment
	course_id	INT UNSIGNED	FK → courses.id (NOT NULL)
	uploaded_by	INT UNSIGNED	FK → users.id (NOT NULL)
	title	VARCHAR(150)	Material title (NOT NULL)
	file_path	VARCHAR(255)	File path or URL (NOT NULL)
	material_type	ENUM	Values: document, link, video (Default: document)
	created_at	TIMESTAMP	Upload timestamp
announcements	id	INT UNSIGNED	Primary Key, Auto Increment
	course_id	INT UNSIGNED	FK → courses.id (NOT NULL)
	author_id	INT UNSIGNED	FK → users.id (NOT NULL)
	title	VARCHAR(150)	Announcement title (NOT NULL)
	body	TEXT	Announcement content (NOT NULL)
	created_at	TIMESTAMP	Creation timestamp
qa_questions	id	INT UNSIGNED	Primary Key, Auto Increment
	course_id	INT UNSIGNED	FK → courses.id (NOT NULL)
	student_id	INT UNSIGNED	FK → users.id (NOT NULL)
	title	VARCHAR(150)	Question title (NOT NULL)
	body	TEXT	Question body/content (NOT NULL)
	is_resolved	TINYINT(1)	1 = Resolved, 0 = Open (Default: 0)
	created_at	TIMESTAMP	Creation timestamp
qa_answers	id	INT UNSIGNED	Primary Key, Auto Increment
	qa_question_id	INT UNSIGNED	FK → qa_questions.id (NOT NULL)
	author_id	INT UNSIGNED	FK → users.id (NOT NULL)

Table Name	Field Name	Data Type	Description / Constraint
	body	TEXT	Answer content (NOT NULL)
	is_endorsed	TINYINT(1)	1 = Endorsed by instructor/TA (Default: 0)
	created_at	TIMESTAMP	Creation timestamp
doubt_sessions	id	INT UNSIGNED	Primary Key, Auto Increment
	course_id	INT UNSIGNED	FK → courses.id (NOT NULL)
	ta_id	INT UNSIGNED	FK → users.id (NOT NULL)
	title	VARCHAR(150)	Session title (NOT NULL)
	scheduled_at	DATETIME	Scheduled date and time (NOT NULL)
	duration_minutes	INT UNSIGNED	Duration in minutes (Default: 60)
	location_or_link	VARCHAR(255)	Physical location or meeting link (NOT NULL)
	max_attendees	INT UNSIGNED	Maximum attendees allowed (Default: 20)
	status	ENUM	Values: scheduled, cancelled, completed (Default: scheduled)
	created_at	TIMESTAMP	Creation timestamp
doubt_session_bookings	id	INT UNSIGNED	Primary Key, Auto Increment
	doubt_session_id	INT UNSIGNED	FK → doubt_sessions.id (NOT NULL)
	student_id	INT UNSIGNED	FK → users.id (NOT NULL)
	booking_status	ENUM	Values: booked, cancelled, attended (Default: booked)
	booked_at	TIMESTAMP	Booking timestamp
platform_settings	id	INT UNSIGNED	Primary Key, Auto Increment
	setting_key	VARCHAR(100)	Unique setting key (NOT NULL)
	setting_value	TEXT	Setting value (NOT NULL)
	setting_type	ENUM	Values: string, integer, decimal, boolean, json (Default: string)
	description	TEXT	Setting description (nullable)
	updated_by	INT UNSIGNED	FK → users.id (nullable, SET NULL on delete)
	created_at	TIMESTAMP	Creation timestamp
	updated_at	TIMESTAMP	Last update timestamp (auto-updated)
platform_announcements	id	INT UNSIGNED	Primary Key, Auto Increment

Table Name	Field Name	Data Type	Description / Constraint
	author_id	INT UNSIGNED	FK → users.id (NOT NULL)
	title	VARCHAR(150)	Announcement title (NOT NULL)
	body	TEXT	Announcement content (NOT NULL)
	audience	ENUM	Values: all, students, instructors, tas (Default: all)
	status	ENUM	Values: draft, published, archived (Default: draft)
	publish_from	DATETIME	Publishing start date (nullable)
	publish_until	DATETIME	Publishing end date (nullable)
	created_at	TIMESTAMP	Creation timestamp
integrity_reports	id	INT UNSIGNED	Primary Key, Auto Increment
	reported_by	INT UNSIGNED	FK → users.id (nullable, SET NULL on delete)
	student_id	INT UNSIGNED	FK → users.id — reported student (nullable)
	quiz_id	INT UNSIGNED	FK → quizzes.id (nullable, SET NULL on delete)
	attempt_id	INT UNSIGNED	FK → attempts.id (nullable, SET NULL on delete)
	report_type	ENUM	Values: suspicious_activity, multiple_logins, copy_paste, rapid_submission, reported_content, policy_violation, other
	severity	ENUM	Values: low, medium, high, critical (Default: medium)
	status	ENUM	Values: open, under_review, resolved, escalated, dismissed (Default: open)
	title	VARCHAR(150)	Report title (NOT NULL)
	description	TEXT	Detailed description (NOT NULL)
	resolution_notes	TEXT	Notes on resolution (nullable)
	resolved_by	INT UNSIGNED	FK → users.id — who resolved it (nullable)
	resolved_at	DATETIME	Resolution timestamp (nullable)
	created_at	TIMESTAMP	Report creation timestamp
audit_logs	id	INT UNSIGNED	Primary Key, Auto Increment
	admin_id	INT UNSIGNED	FK → users.id — admin who acted (nullable, SET NULL on delete)

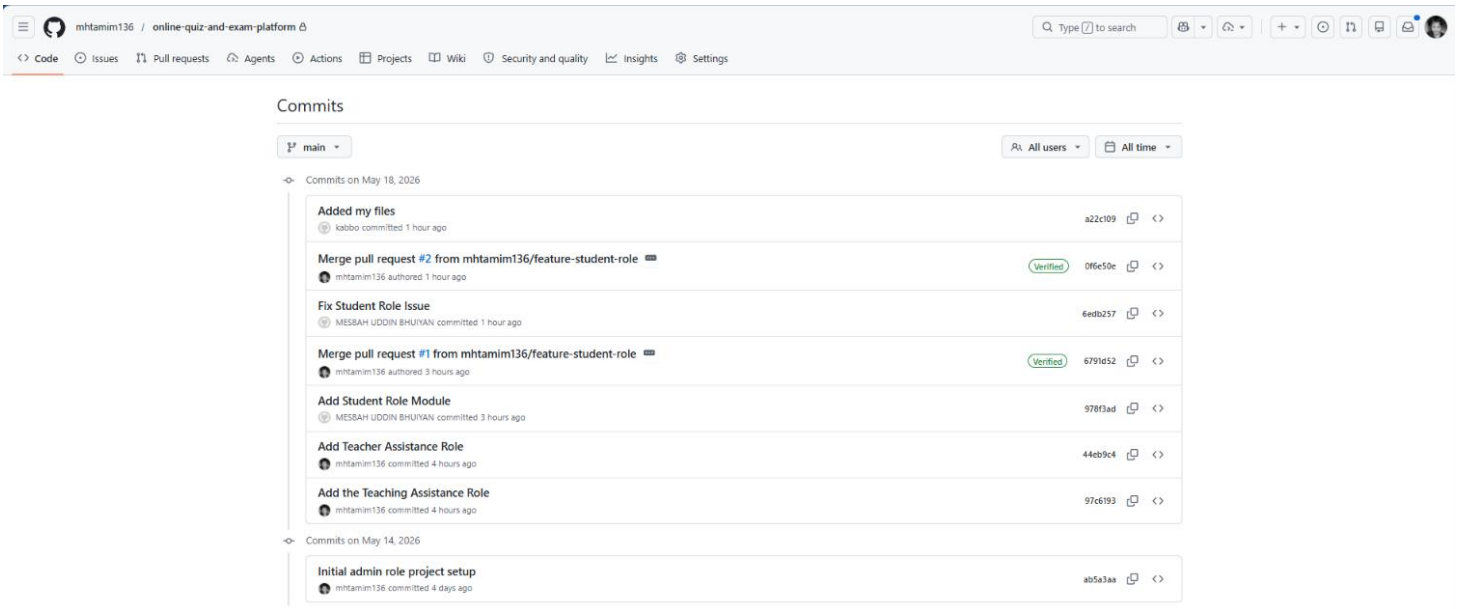
Table Name	Field Name	Data Type	Description / Constraint
	action	VARCHAR(100)	Action performed (e.g., create_user, deactivate_user)
	entity_type	VARCHAR(100)	Entity type affected (e.g., users, courses)
	entity_id	INT UNSIGNED	ID of the affected entity (nullable)
	ip_address	VARCHAR(45)	IP address of the actor (nullable)
	user_agent	VARCHAR(255)	Browser/client info (nullable)
	details	TEXT	JSON details of the action (nullable)
	created_at	TIMESTAMP	Log creation timestamp

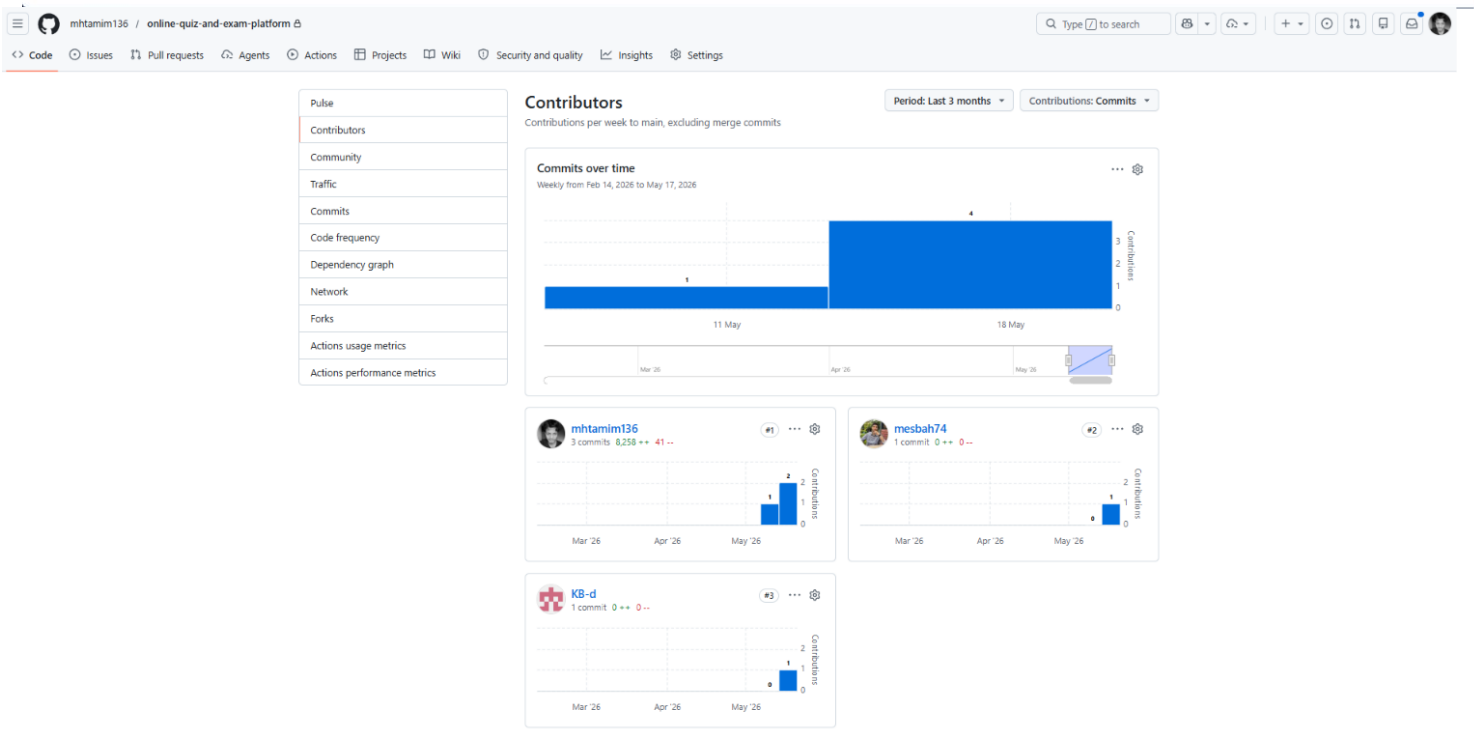
Chapter 2: Git Contribution & Task Allocation

2.1 GitHub Repository Link

Repository URL: https://github.com/mhtamim136/online-quiz-and-exam-platform.git

2.2 Git Contribution Screenshot





Chapter 3: Features, Source Code & Screenshots

Feature: Role-Based Login Redirect

Description

Purpose: Users log in with email and password. After successful login, the system redirects them to the correct dashboard based on their role: Admin, Student, Instructor, or TA.

User interaction: The user opens the login page, enters email and password, then submits the form.

Why needed: Login protects the platform and ensures each user can access only the pages allowed for their role.

Relevant Source Code

File: controllers/AuthController.php

```php
function authHandleLogin($conn)
{
    $email = isset($_POST['email']) ? trim($_POST['email']) : '';
    $password = isset($_POST['password']) ? trim($_POST['password']) : '';
    $errors = array();
    $old = array('email' => $email);

    if ($email === '') {
        $errors[] = 'Email is required.';
    } elseif (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
        $errors[] = 'Please enter a valid email address.';
    }

    if ($password === '') {
        $errors[] = 'Password is required.';
    }

    if (count($errors) > 0) {
        authShowLogin($errors, $old);
        return;
    }

    $user = userFindByEmail($conn, $email);

    if (!$user) {
        $errors[] = 'No user account was found for this email.';
        authShowLogin($errors, $old);
        return;
    }

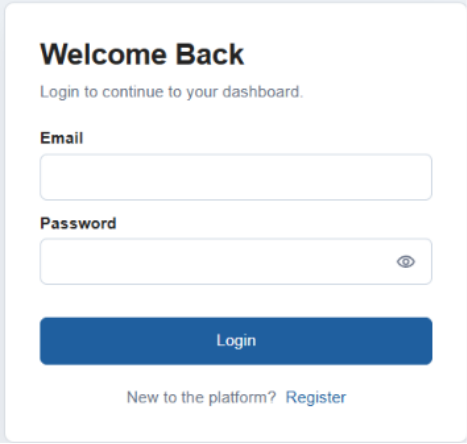
    if ((int)$user['is_active'] !== 1) {
        $errors[] = 'This account is inactive. Please contact the platform admin.';
        authShowLogin($errors, $old);
        return;
    }
```

Code Explanation

- `authHandleLogin()` handles the login form submission.
- `$email` and `$password` collect user input from `$_POST`.
- `$errors` stores validation messages.
- The email validation checks that email is not empty and has a valid format.
- The password validation checks that the password field is not empty.

- If validation fails, `authShowLogin()` reloads the login page with errors.
- `userFindByEmail()` searches the `users` table using the submitted email.
- If no user is found, the login page shows an account-not-found message.
- The `is_active` check blocks inactive users from logging in.
- `password_verify()` checks the plain password against the hashed password stored in the database.
- If the password is correct, user details are saved in `$_SESSION`.
- `authRedirectByRole()` sends the user to the correct dashboard based on role.
- `userFindByEmail()` uses a prepared query with `?`, which helps avoid SQL injection.

Screenshots



Feature: Student and Instructor Registration Request

Description

Purpose: New users can register as Student or Instructor. Student accounts become active immediately, but Instructor accounts stay pending until an Admin approves them.

User interaction: The user opens the registration page, selects account type, fills required information, and submits the form.

Why needed: Students need simple self-registration, while instructors need approval because they can manage academic content.

Relevant Source Code

File: controllers/AuthController.php

```php
function authHandleRegister($conn)
{
    $old = array(
        'name' => isset($_POST['name']) ? trim($_POST['name']) : '',
        'username' => isset($_POST['username']) ? trim($_POST['username']) : '',
        'email' => isset($_POST['email']) ? trim($_POST['email']) : '',
        'phone' => isset($_POST['phone']) ? trim($_POST['phone']) : '',
        'role' => isset($_POST['role']) ? trim($_POST['role']) : 'student',
        'student_id' => isset($_POST['student_id']) ? trim($_POST['student_id']) : '',
        'program' => isset($_POST['program']) ? trim($_POST['program']) : '',
        'department' => isset($_POST['department']) ? trim($_POST['department']) : '',
        'bio' => isset($_POST['bio']) ? trim($_POST['bio']) : '',
        'password' => isset($_POST['password']) ? trim($_POST['password']) : '',
        'confirm_password' => isset($_POST['confirm_password']) ? trim($_POST['confirm_password']) :
    );

    $old['password_hash'] = password_hash($password, PASSWORD_DEFAULT);

    if ($old['role'] === 'student') {
        $old['approval_status'] = 'approved';
        $old['is_active'] = 1;
        $old['bio'] = '';
    } else {
        $old['approval_status'] = 'pending';
        $old['is_active'] = 0;
        $old['student_id'] = '';
        $old['program'] = '';
    }

    userCreateRegisteredUser($conn, $old);
    redirect('login.php');
}
```

File : models/UserModel.php

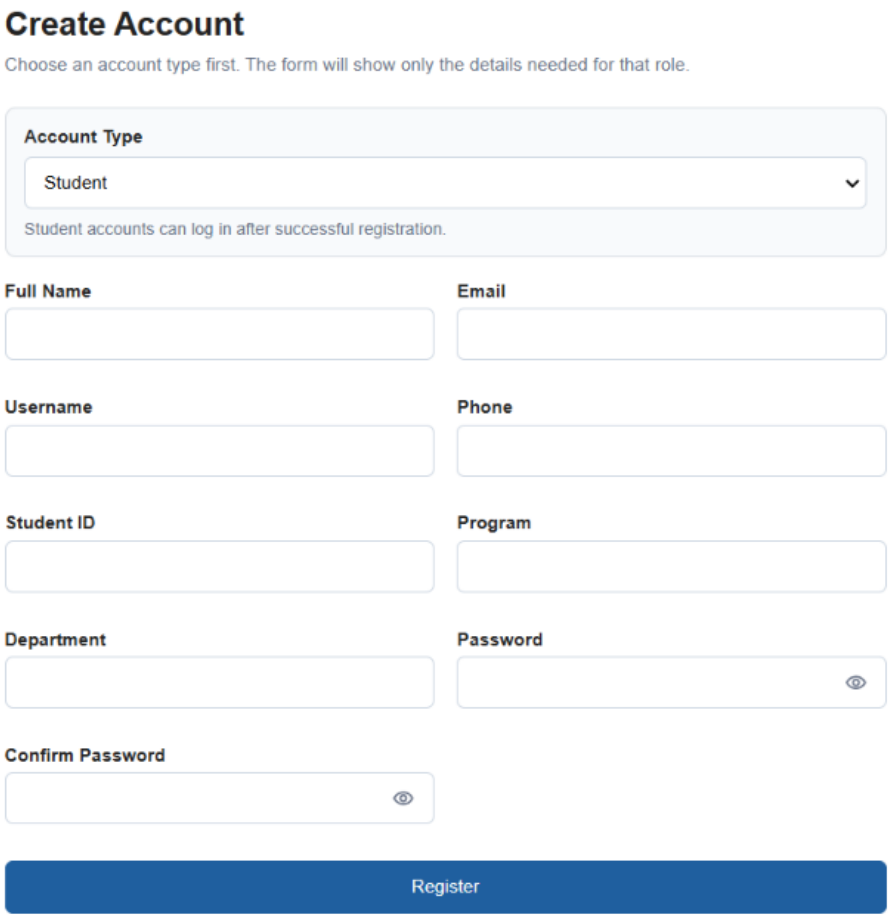
```php
function userCreateRegisteredUser($conn, $data)
{
    $sql = "INSERT INTO users
            (username, name, email, password_hash, phone, role, student_id, program, department, bio,
            approval_status, is_active)
            VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)";
    return dbExecute($conn, $sql, 'sssssssssssi', array(
        $data['username'], $data['name'], $data['email'], $data['password_hash'],
        $data['phone'], $data['role'], $data['student_id'], $data['program'],
        $data['department'], $data['bio'], $data['approval_status'], (int)$data['is_active']
    ));
}
```

Code Explanation

- `authHandleRegister()` handles registration form submission.
- `$old` collects all registration fields from `$_POST`.
- The role defaults to `student` if no role is selected.
- `password_hash()` converts the plain password into a secure hash.
- If the user registers as student, `approval_status` becomes `approved`.
- Student accounts also get `is_active = 1`, so they can log in immediately.
- If the user registers as instructor, `approval_status` becomes `pending`.

- Instructor accounts get `is_active = 0`, so they cannot log in until admin approval.
- `student_id` and `program` are cleared for instructor accounts because those fields belong to students.
- `userCreateRegisteredUser()` inserts the new account into the `users` table.
- The prepared INSERT query saves all account details safely.
- After successful registration, the user is redirected to the login page.

Screenshot(s)



Feature: **Admin Dashboard**

Description

Admins see platform totals such as users by role, active courses, attempts today, and pending instructor approvals. After admin login, the Admin Dashboard paperchains need a quick overview of platform activity and pending tasks.

Relevant Source Code

File: `controllers/AdminController.php`

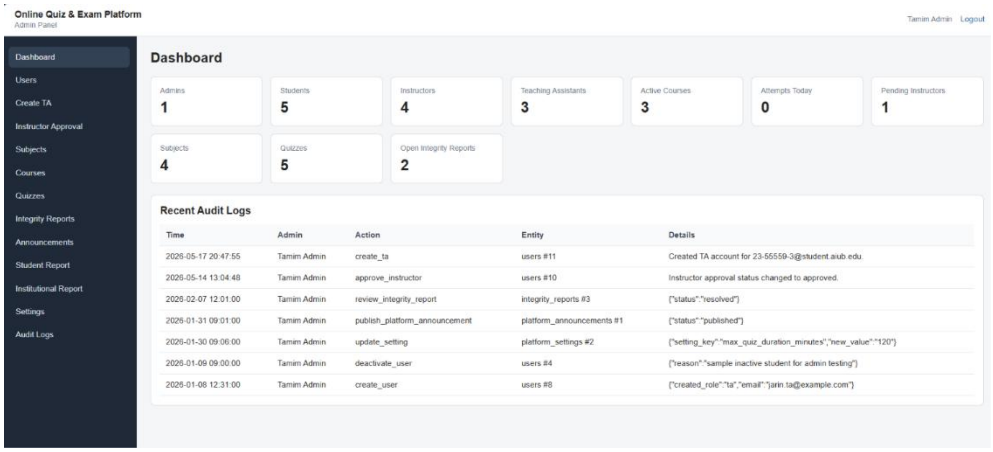
```
function adminDashboard($conn)
{
    $pageTitle = 'Dashboard';
    $stats = adminGetDashboardStats($conn);
    adminView('dashboard', compact('pageTitle', 'stats'));
}
```

```
function adminGetDashboardStats($conn)
{
    $stats = array();
    $stats['users_by_role'] = dbSelectAll($conn, "SELECT role, COUNT(*) AS total FROM users GROUP BY role ORDER BY role ASC");
    $stats['active_courses'] = dbSelectOne($conn, "SELECT COUNT(*) AS total FROM courses WHERE status = 'active'");
    $stats['attempts_today'] = dbSelectOne($conn, "SELECT COUNT(*) AS total FROM attempts WHERE DATE(started_at) = CURDATE()");
    $stats['pending_instructors'] = dbSelectOne($conn, "SELECT COUNT(*) AS total FROM users WHERE role = 'instructor' AND approval_status = 'pending'");
    return $stats;
}
```

Code Explanation

- `adminDashboard()` loads dashboard statistics from the model.
- `users_by_role` groups users by role.
- `active_courses` counts courses currently marked active.
- `attempts_today` counts attempts that started today.
- `pending_instructors` counts instructor accounts waiting for admin approval

Screenshot(s)



Feature: Manage User Accounts

Description

Admins can search, view, activate, deactivate, and change roles for users.Admin opens Users, filters/searches accounts, and uses action buttons or role dropdowns.Central account control is required for platform security and access management.

Relevant Source Code

File: controllers/AdminController.php

```php
function adminUsers($conn)
{
    $keyword = isset($_GET['keyword']) ? trim($_GET['keyword']) : '';
    $role = isset($_GET['role']) ? trim($_GET['role']) : '';
    $active = isset($_GET['active']) ? trim($_GET['active']) : '';
    $users = userGetAll($conn, $keyword, $role, $active);
    adminView('users', compact('users', 'keyword', 'role', 'active'));
}
```

File : models/UserModel.php

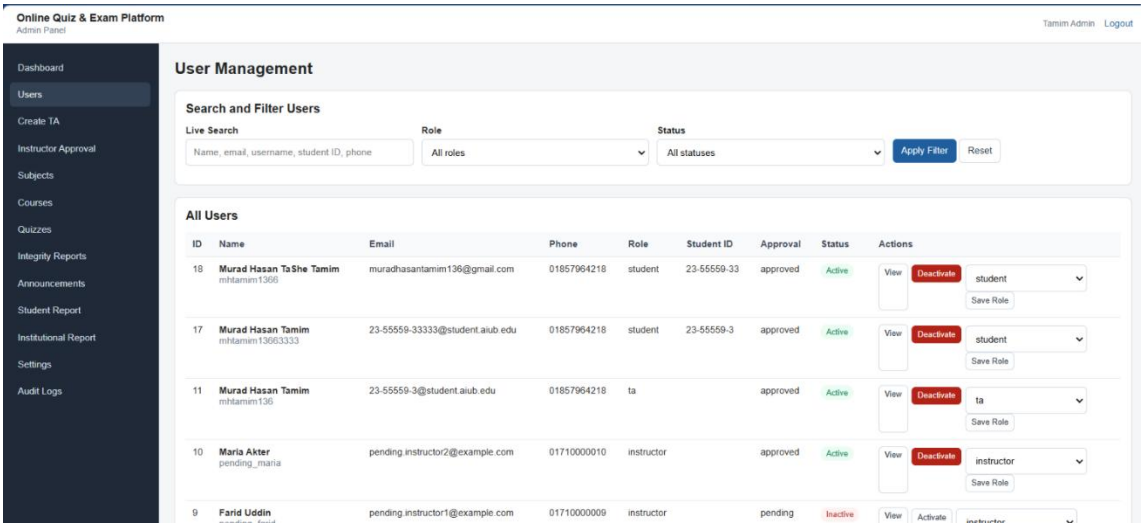
```php
function userSetActiveStatus($conn, $user_id, $is_active)
{
    $sql = "UPDATE users SET is_active = ? WHERE id = ?";
    return dbExecute($conn, $sql, 'ii', array((int)$is_active, (int)$user_id))
}

function userChangeRole($conn, $user_id, $role)
{
    $sql = "UPDATE users SET role = ? WHERE id = ?";
    return dbExecute($conn, $sql, 'si', array($role, (int)$user_id));
}
```

Code Explanation

- adminUsers() reads search and filter values from the URL.
- userGetAll() returns matching users.
- userSetActiveStatus() updates whether the account can log in.
- userChangeRole() changes the user's platform role.
- Admin controller actions also call logAdminAction() after important changes.

Screenshot



Feature: Approve or Reject Instructor Requests

Description

Admins approve or reject new instructor registrations.Admin opens Instructor Approval and clicks Approve or Reject.Instructor access should be controlled before they manage courses and quizzes.

Relevant Source Code

File: controllers/AdminController.php

```
```php
function adminApproveInstructor($conn)

    adminHandleInstructorDecision($conn, 'approved', 1, 'approve_instructor', 'Instructor approved successfully.');

function adminRejectInstructor($conn)

    adminHandleInstructorDecision($conn, 'rejected', 0, 'reject_instructor', 'Instructor rejected successfully.');

```

models/UserModel.php` - PHP

```php
function userUpdateInstructorApproval($conn, $user_id, $approval_status, $is_active)

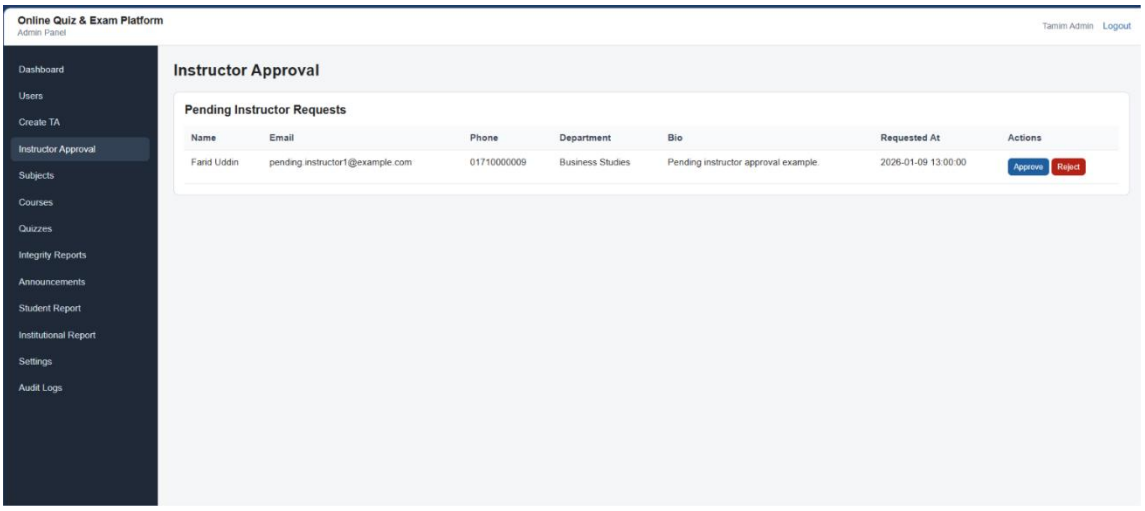
    $sql = "UPDATE users
            SET approval_status = ?, is_active = ?
            WHERE id = ? AND role = 'instructor'";
    return dbExecute($conn, $sql, 'sii', array($approval_status, (int)$is_active, (int)$user_id));

```
```

Code Explanation

- Approving sets `approval_status` to `approved` and activates the account.
- Rejecting sets `approval_status` to `rejected` and keeps the account inactive.
- `WHERE role = 'instructor'` prevents this approval action from affecting other roles.
- The admin controller logs the decision in the audit log.

Screenshot



Feature: Create TA Accounts Directly

Description

Admins can create TA accounts without public registration.Admin opens Create TA, enters name, username, email, phone, department, bio, and password.TAs are staff/support users and should be created by Admin.

Relevant Source Code

File: controllers/AdminController.php

```
function adminCreateTa($conn)
{
    if (isPost()) {
        $old['password_hash'] = password_hash($password, PASSWORD_DEFAULT);
        $new_id = userCreateTa($conn, $old);
        logAdminAction($conn, 'create_ta', 'users', (int)$new_id, 'Created TA account for ' . $old['email'] . '.');
        redirect('admin.php?action=create_ta');
    }
}
```

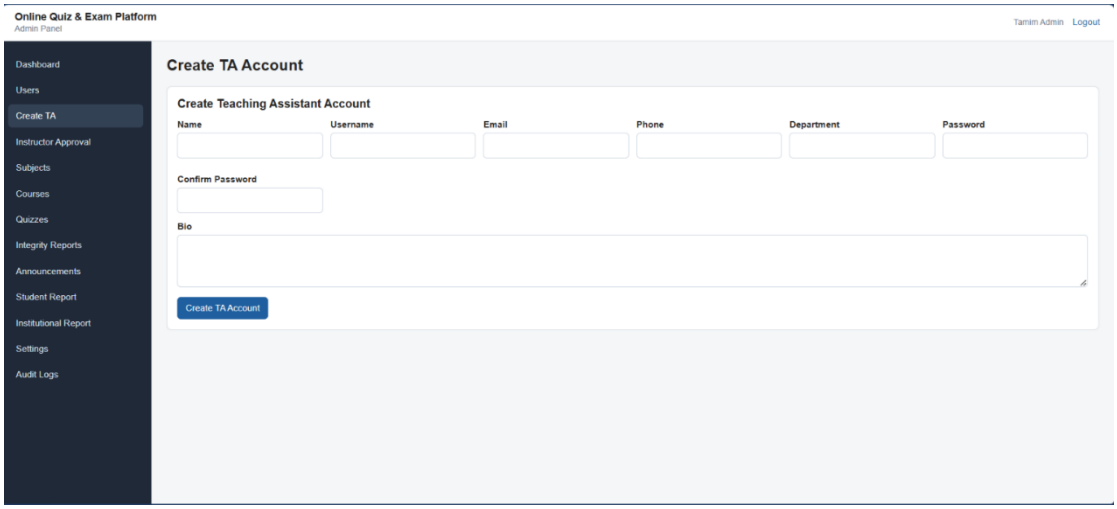
`models/UserModel.php` - PHP

```php
function userCreateTa($conn, $data)
{
    $sql = "INSERT INTO users
            (username, name, email, password_hash, phone, role, department, bio, approval_status, is_active)
            VALUES (?, ?, ?, ?, ?, 'ta', ?, ?, 'approved', 1)";
    return dbExecute($conn, $sql, 'sssssss', array($data['username'], $data['name'], $data['email'], $data['password_hash'], $data['phone'], $data['department'], $data['bio']));
}
```

Code Explanation

- The controller validates the submitted TA form.
- password_hash() securely stores the password.
- userCreateTa() inserts the user with `role = 'ta'`.
- The TA account is automatically approved and active.
- logAdminAction()`records the account creation.

Screenshot



Feature: View and Filter All Courses

Description

Admins can view active, draft, and archived courses and filter by subject or instructor.Admin opens Courses and uses filter dropdowns.Admins need oversight of the whole course catalog.

Relevant Source Code

File: controllers/AdminController.php

```
function adminCourses($conn)
{
    $subject_id = isset($_GET['subject_id']) ? (int)$_GET['subject_id'] : 0;
    $instructor_id = isset($_GET['instructor_id']) ? (int)$_GET['instructor_id'] : 0;
    $status = isset($_GET['status']) ? trim($_GET['status']) : '';

    $courses = courseGetAll($conn, $subject_id, $instructor_id, $status);
    adminView('courses', compact('courses', 'subject_id', 'instructor_id', 'status'));
}
```

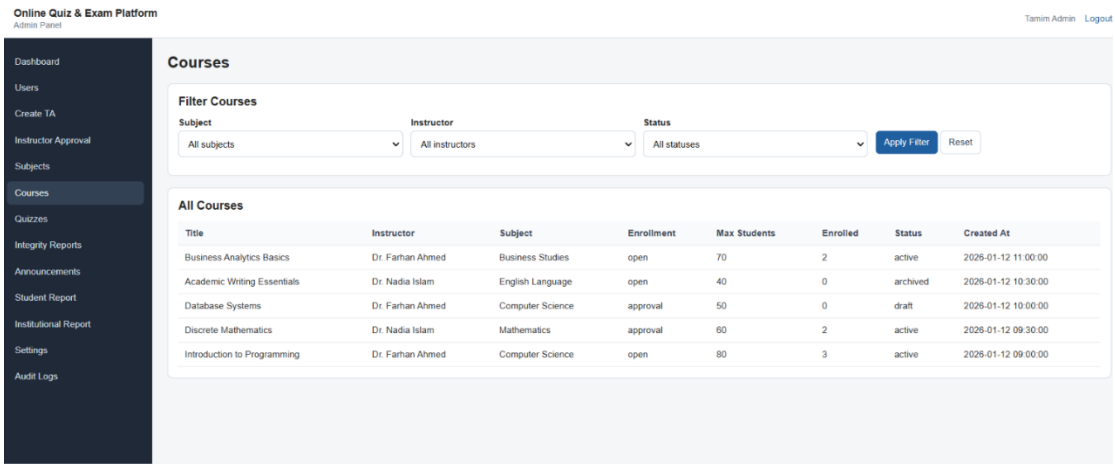
`models/CourseModel.php` - PHP

```php
function courseGetAll($conn, $subject_id, $instructor_id, $status)
{
    $sql = "SELECT courses.*, subjects.name AS subject_name, users.name AS instructor_name,
                   (SELECT COUNT(*) FROM enrollments WHERE enrollments.course_id = courses.id) AS enrolled_count
            FROM courses
            INNER JOIN subjects ON subjects.id = courses.subject_id
            INNER JOIN users ON users.id = courses.instructor_id";
}
```

Code Explanation

- The controller reads optional subject, instructor, and status filters.
- `courseGetAll()` joins courses with subjects and instructors for readable display.
- The subquery counts enrollments per course.
- Additional `WHERE` conditions are added only when filters are selected.

Screenshot



Feature: Manage Subject

Description

Admins can add, rename, and delete subjects.Admin opens Subjects, submits add/edit forms, or deletes unused subjects.Subjects organize courses and reports.

Relevant Source Code

File: `models/SubjectModel.php`

```php
function subjectCreate($conn, $name, $description)
{
    $sql = "INSERT INTO subjects (name, description) VALUES (?, ?)";
    return dbExecute($conn, $sql, 'ss', array($name, $description));
}

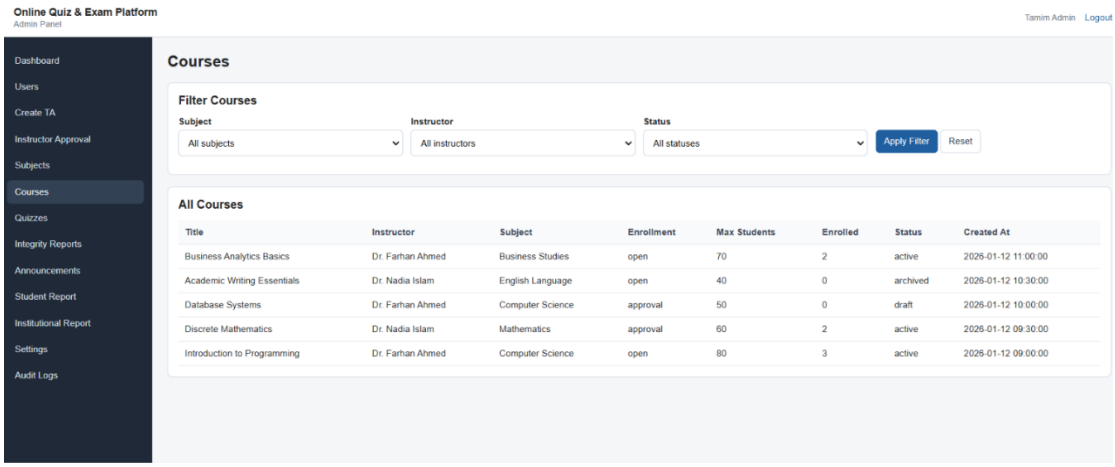
function subjectUpdate($conn, $id, $name, $description)
{
    $sql = "UPDATE subjects SET name = ?, description = ? WHERE id = ?";
    return dbExecute($conn, $sql, 'ssi', array($name, $description, (int)$id));
}

function subjectDelete($conn, $id)
{
    $sql = "DELETE FROM subjects WHERE id = ?";
    return dbExecute($conn, $sql, 'i', array((int)$id));
}
```

Code Explanation

- `subjectCreate()` inserts a new subject.
- `subjectUpdate()` changes subject name and description.
- `subjectDelete()` removes a subject by id.
- The controller checks `subjectCourseCount()` before delete so subjects used by courses are not removed.

Screenshot



Feature: View All Quizzes

Description

Admins can view quizzes across the platform and filter by course, status, or type.

Admin opens Quizzes and applies filters.Admins need oversight of quiz activity and attempt counts.

Relevant Source Code

File: `models/SubjectModel.php`

```
function adminQuizzes($conn)
{
    $course_id = isset($_GET['course_id']) ? (int)$_GET['course_id'] : 0;
    $status = isset($_GET['status']) ? trim($_GET['status']) : '';
    $quiz_type = isset($_GET['quiz_type']) ? trim($_GET['quiz_type']) : '';

    $quizzes = quizGetAll($conn, $course_id, $status, $quiz_type);
    adminView('quizzes', compact('quizzes', 'course_id', 'status', 'quiz_type'));
}
```

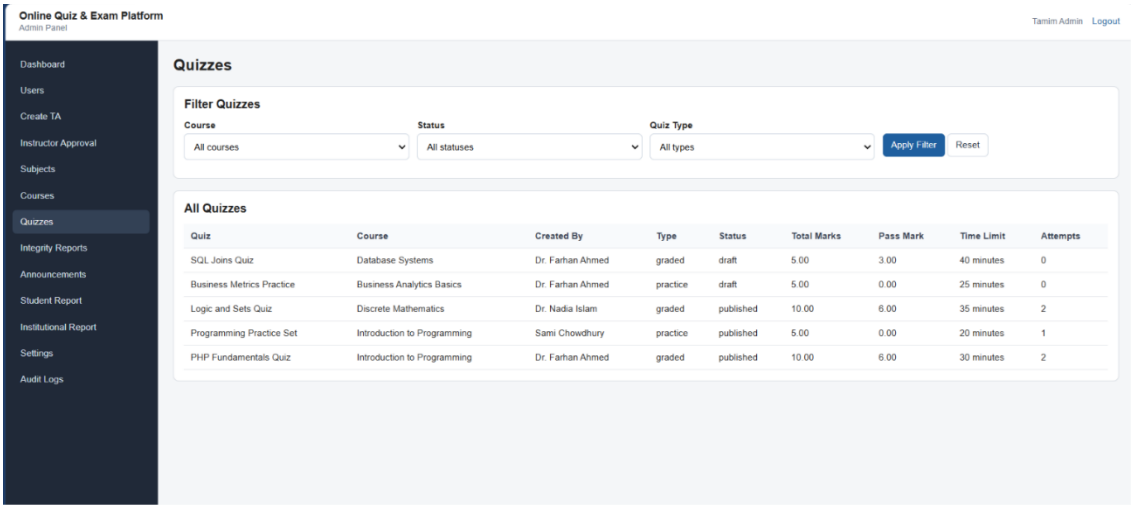
`models/QuizModel.php` - PHP

```php
function quizGetAll($conn, $course_id, $status, $quiz_type)
{
    $sql = "SELECT quizzes.*, courses.title AS course_title, users.name AS creator_name,
                   (SELECT COUNT(*) FROM attempts WHERE attempts.quiz_id = quizzes.id) AS attempt_count
            FROM quizzes
            INNER JOIN courses ON courses.id = quizzes.course_id
            INNER JOIN users ON users.id = quizzes.created_by";
}
```

Code Explanation

- The controller reads quiz filters from the URL.
- The model joins quizzes with courses and creator users.
- The attempt-count subquery counts attempts per quiz.
- Filters can narrow by course, quiz status, and quiz type.

Screenshot



Feature: Handle Integrity Reports

Description

Admins can review reported content or academic integrity flags and mark them as resolved or escalated. Admin opens Integrity Reports, selects a report, reads details, enters resolution notes, and updates status. Academic integrity issues require review and tracking.

Relevant Source Code

File: `models/SubjectModel.php`

```
function adminIntegrityReports($conn)
{
    if (isPost()) {
        $report_id = isset($_POST['report_id']) ? (int)$_POST['report_id'] : 0;
        $new_status = adminClean('status');
        $resolution_notes = adminClean('resolution_notes');
        integrityReportUpdateStatus($conn, $report_id, $new_status, $resolution_notes, currentAdminId());
    }

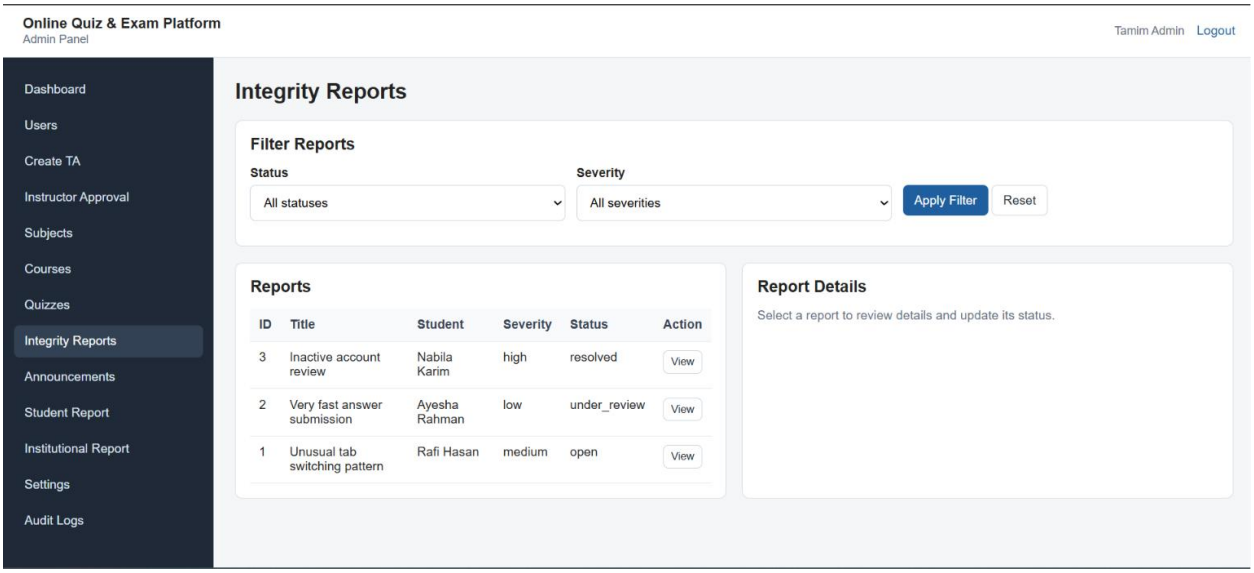
    $reports = integrityReportGetAll($conn, $status, $severity);
    $selectedReport = integrityReportFindById($conn, (int)$_GET['report_id']);
}
```

Code Explanation

- The controller handles report status update on POST.
- It reads report id, new status, and resolution notes.

- `integrityReportUpdateStatus()` updates the report status.
- It stores who resolved it using `resolved_by`.
- `resolved_at = NOW()` records the decision time.

Screenshot



Feature: Platform-Wide Analytics

Description

Purpose: Admins can see enrollments per subject, quiz pass rates, and most active instructors.User interaction: Admin opens Institutional Report.Why needed: Platform analytics help administrators understand usage and performance.Relevant Source Code

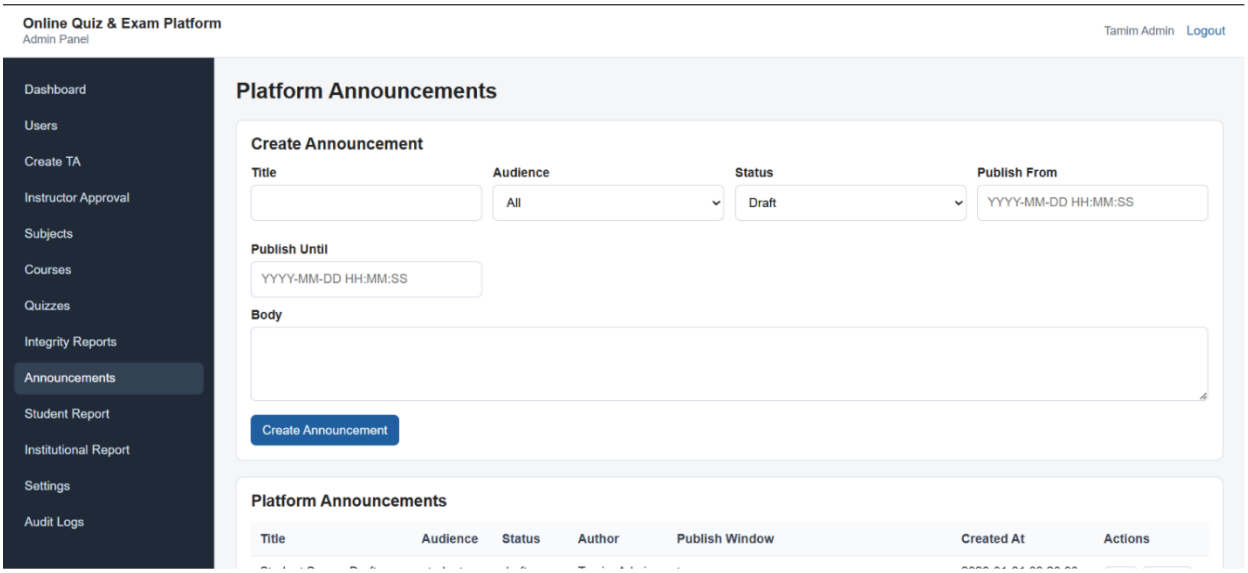
File: ` `models/AdminModel.php`

```
function adminGetPassRatesPerSubject($conn)
{
    $sql = "SELECT subjects.name AS subject_name,
                   COUNT(attempts.id) AS total_attempts,
                   SUM(CASE WHEN attempts.score >= quizzes.pass_mark THEN 1 ELSE 0 END) AS passed_attempts
            FROM subjects
            LEFT JOIN courses ON courses.subject_id = subjects.id
            LEFT JOIN quizzes ON quizzes.course_id = courses.id
            LEFT JOIN attempts ON attempts.quiz_id = quizzes.id
            GROUP BY subjects.id, subjects.name";
    return dbSelectAll($conn, $sql);
}
```

Code Explanation

- The query starts with all subjects.
- It joins courses, quizzes, and attempts.
- `COUNT(attempts.id)` counts attempts per subject.
- The `SUM(CASE...)` expression counts attempts where score is greater than or equal to the pass mark.
- Grouping by subject creates one analytics row per subject.

Screenshot



Feature: Manage Platform Announcements

Description

Admins can create, edit, publish, archive, and schedule platform-wide announcements. Admin opens Announcements, fills title/body/audience/status/date range, and submits. Admins need a way to communicate platform notices to all or selected user groups.

File: models/PlatformAnnouncementModel.php

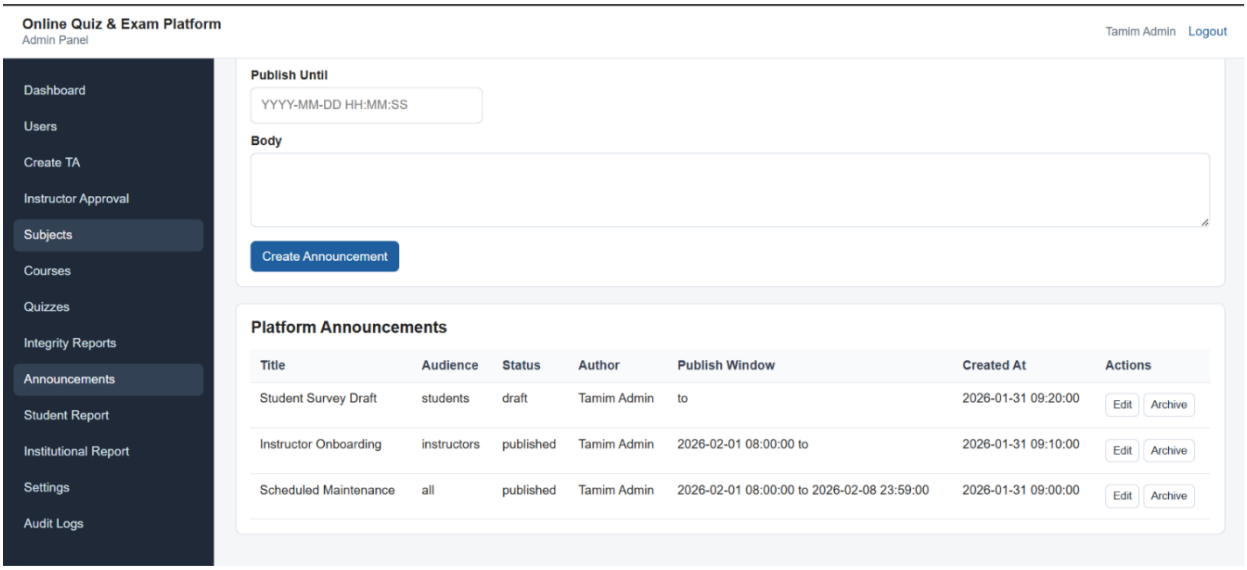
```
function platformAnnouncementCreate($conn, $author_id, $title, $body, $audience, $status, $publish_from, $publish_until)
{
    $sql = "INSERT INTO platform_announcements
            (author_id, title, body, audience, status, publish_from, publish_until)
            VALUES (?, ?, ?, ?, ?, ?, ?)";
    return dbExecute($conn, $sql, 'isssss', array((int)$author_id, $title, $body, $audience, $status, $publish_from, $publish_until));
}

function platformAnnouncementArchive($conn, $id)
{
    $sql = "UPDATE platform_announcements SET status = 'archived' WHERE id = ?";
    return dbExecute($conn, $sql, 'i', array((int)$id));
}
```

Code Explanation

- platformAnnouncementCreate() inserts an announcement with author id, content, audience, status, and publish dates.
- audience controls which group the announcement targets.
- status controls draft/published/archived state.
- platformAnnouncementArchive() changes status to archived instead of deleting the announcement.

Screenshot



Feature: Student Academic Report

Description

Purpose: Admins can search a student and view course enrollments, attempt counts, and average score.

User interaction: Admin opens Student Report and searches by student id, name, or email.

Why needed: Admins need per-student academic visibility for support and reporting.

File: controllers/AdminController.php

```
function adminStudentReport($conn)
{
    $keyword = isset($_GET['keyword']) ? trim($_GET['keyword']) : '';
    $student_id = isset($_GET['student_id']) ? (int)$_GET['student_id'] : 0;

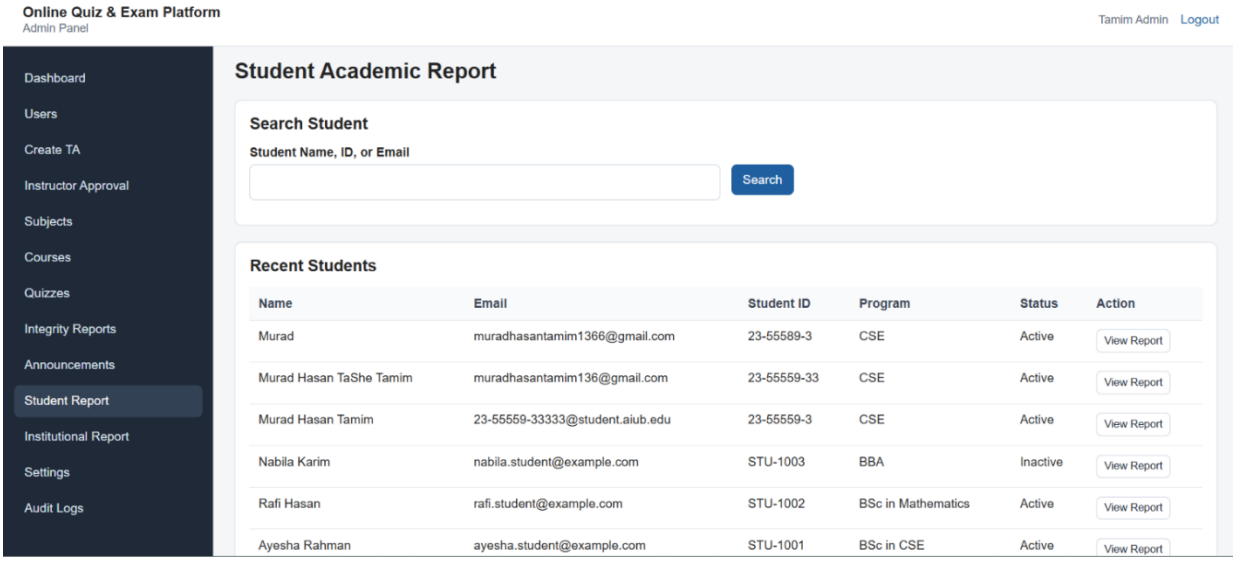
    if ($keyword !== '') {
        $students = userGetStudentsByKeyword($conn, $keyword);
    }

    if ($student_id > 0) {
        $enrollments = adminGetStudentEnrollments($conn, $student_id);
        $summary = adminGetStudentAttemptSummary($conn, $student_id);
        $attempts = adminGetStudentAttempts($conn, $student_id);
    }
}
---
```

Code Explanation

- The controller reads search keyword and selected student id.
- userGetStudentsByKeyword() finds matching students.
- When a student is selected, enrollments, attempt summary, and attempt details are loaded.
- adminGetStudentAttemptSummary()counts attempts and calculates average score for that student.

Screenshot



Feature: Institutional Report

Description

Purpose: Admins can search a student and view course enrollments, attempt counts, and average score.

User interaction: Admin opens Student Report and searches by student id, name, or email.

Why needed: Admins need per-student academic visibility for support and reporting.

File: controllers/AdminController.php

```
function adminInstitutionalReport($conn)
{
    $totals = adminGetInstitutionalTotals($conn);
    $passRates = adminGetPassRatesPerSubject($conn);
    $enrollments = adminGetEnrollmentsPerSubject($conn);
    $activeInstructors = adminGetMostActiveInstructors($conn);
    adminView('institutional_report', compact('totals', 'passRates', 'enrollments', 'activeInstructors'));
}
```

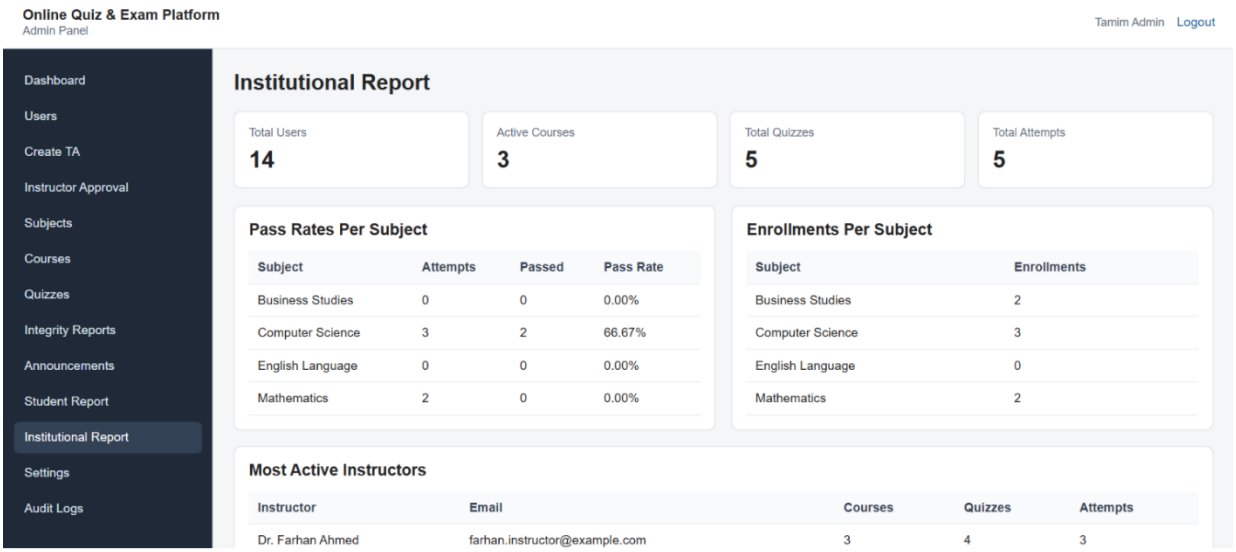
`models/AdminModel.php` - PHP

```php
function adminGetInstitutionalTotals($conn)
{
    return array(
        'users' => dbSelectOne($conn, "SELECT COUNT(*) AS total FROM users"),
        'active_courses' => dbSelectOne($conn, "SELECT COUNT(*) AS total FROM courses WHERE status = 'active'"),
        'quizzes' => dbSelectOne($conn, "SELECT COUNT(*) AS total FROM quizzes"),
        'attempts' => dbSelectOne($conn, "SELECT COUNT(*) AS total FROM attempts")
    );
}
```

Code Explanation

- The controller collects several report datasets.
- `adminGetInstitutionalTotals()` returns overall counts.
- Other model functions calculate pass rates, enrollments per subject, and active instructors.
- The view displays these datasets in cards and tables.

Screenshot



Feature: Platform-Wide Policies and Settings

Description

Purpose: Admins can update platform settings such as maximum quiz duration, default maximum students per course, and at-risk score threshold.

User interaction: Admin opens Settings, edits a setting value, and clicks Save.

Why needed: Platform behavior should be configurable without changing source code.

File: controllers/AdminController.php

```
function adminSettings($conn)
{
    if (isPost()) {
        $setting_id = isset($_POST['setting_id']) ? (int)$_POST['setting_id'] : 0;
        $setting_value = adminClean('setting_value');
        platformSettingUpdate($conn, $setting_id, $setting_value, currentAdminId());
    }

    $settings = platformSettingGetAll($conn);
    adminView('settings', compact('settings'));
}
```

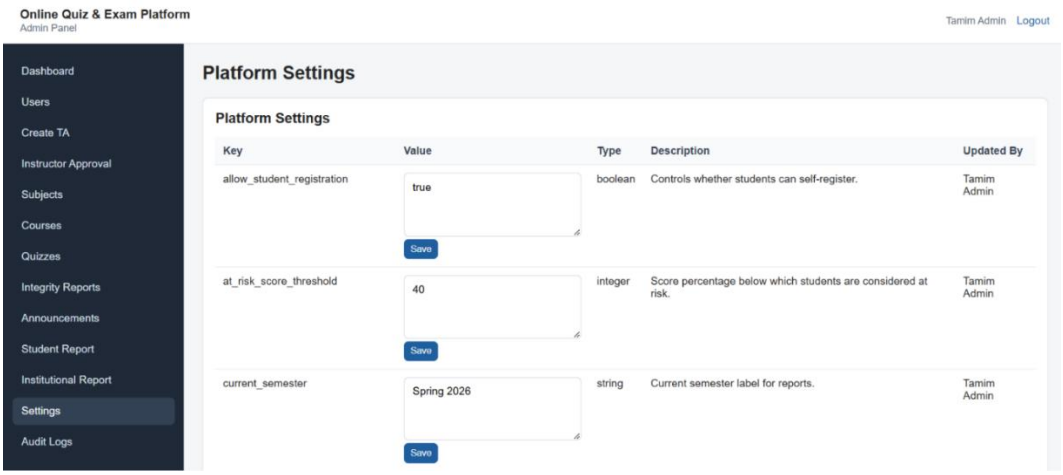
`models/PlatformSettingModel.php` - PHP

```php
function platformSettingUpdate($conn, $id, $setting_value, $updated_by)
{
    $sql = "UPDATE platform_settings
            SET setting_value = ?, updated_by = ?
            WHERE id = ?";
    return dbExecute($conn, $sql, 'sii', array($setting_value, (int)$updated_by, (int)$id));
}
```

Code Explanation

- The controller receives the setting id and new value.
- It validates the value according to the setting type.
- `platformSettingUpdate()` saves the new value.
- `updated_by` records which admin changed the setting.

Screenshot



Feature: Audit Log

Description

Purpose: Admins can view significant admin actions.

User interaction: Admin opens Audit Logs and filters by action or admin.

Why needed: Audit logs provide accountability for sensitive administrative changes.

File: config/helpers.php`

```
function logAdminAction($conn, $action, $entity_type, $entity_id, $details)
{
    $admin_id = isset($_SESSION['user_id']) ? (int)$_SESSION['user_id'] : null;
    $ip_address = isset($_SERVER['REMOTE_ADDR']) ? $_SERVER['REMOTE_ADDR'] : '';
    $user_agent = isset($_SERVER['HTTP_USER_AGENT']) ? substr($_SERVER['HTTP_USER_AGENT'], 0, 255) : '';

    $sql = "INSERT INTO audit_logs (admin_id, action, entity_type, entity_id, ip_address, user_agent, details)
            VALUES (?, ?, ?, ?, ?, ?, ?)";
    return dbExecute($conn, $sql, 'ississs', array($admin_id, $action, $entity_type, $entity_id, $ip_address, $user_agent, $details));
}
```

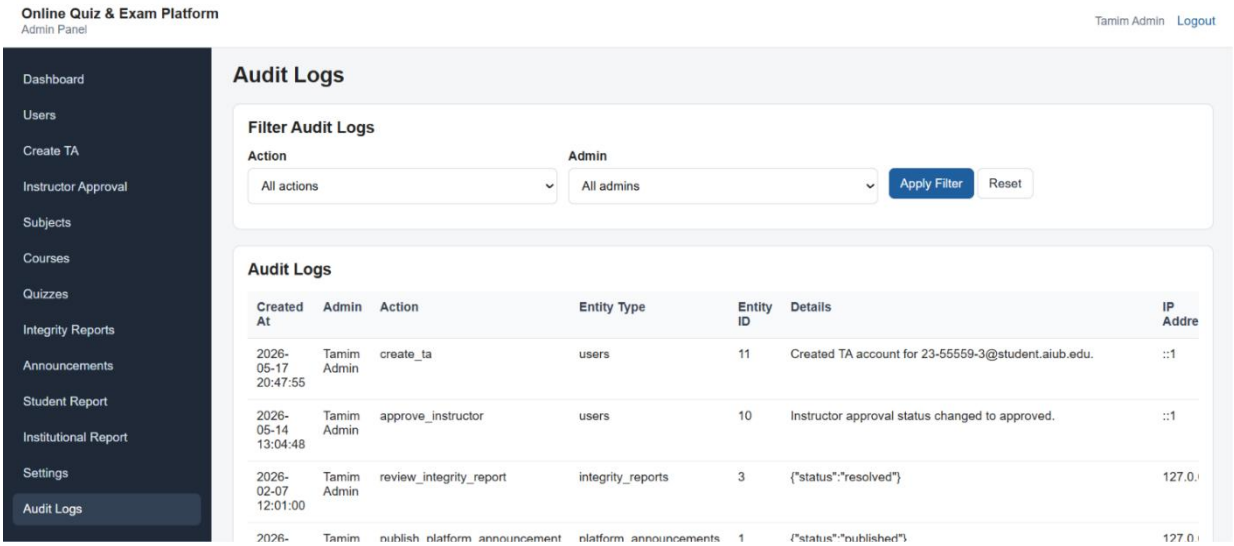
`models/AuditLogModel.php` - PHP

```php
function auditLogGetAll($conn, $action, $admin_id)
{
    $sql = "SELECT audit_logs.*, users.name AS admin_name
            FROM audit_logs
            LEFT JOIN users ON users.id = audit_logs.admin_id";
    return dbSelectAll($conn, $sql, $types, $params);
}
```

Code Explanation

- logAdminAction()records action name, entity type, entity id, IP address, browser user agent, and details.
- Admin actions such as create TA, approve instructor, update settings, and role changes call this helper.
- auditLogGetAll() loads audit records and joins admin names.
- Filters can limit logs by action or admin.

Screenshot



Teaching Assistant Features

Feature: Manage Profile

Description

Purpose: A Teaching Assistant logs in with email and password, then manages personal profile details such as name, phone, department, bio, profile picture, and password**.**

User interaction: The TA signs in from the login page. After login, the TA opens the Profile page from the TA sidebar and submits profile/password forms.

Why needed: The platform needs authenticated TA accounts so assigned course support actions are linked to the correct user.

File: controllers/AuthController.php

```
function authHandleLogin($conn)
{
    $email = isset($_POST['email']) ? trim($_POST['email']) : '';
    $password = isset($_POST['password']) ? trim($_POST['password']) : '';

    $user = userFindByEmail($conn, $email);

    if ((int)$user['is_active'] !== 1) {
        $errors[] = 'This account is inactive. Please contact the platform admin.';
        authShowLogin($errors, $old);
        return;
    }

    if (!password_verify($password, $user['password_hash'])) {
        $errors[] = 'Incorrect password.';
        authShowLogin($errors, $old);
        return;
    }

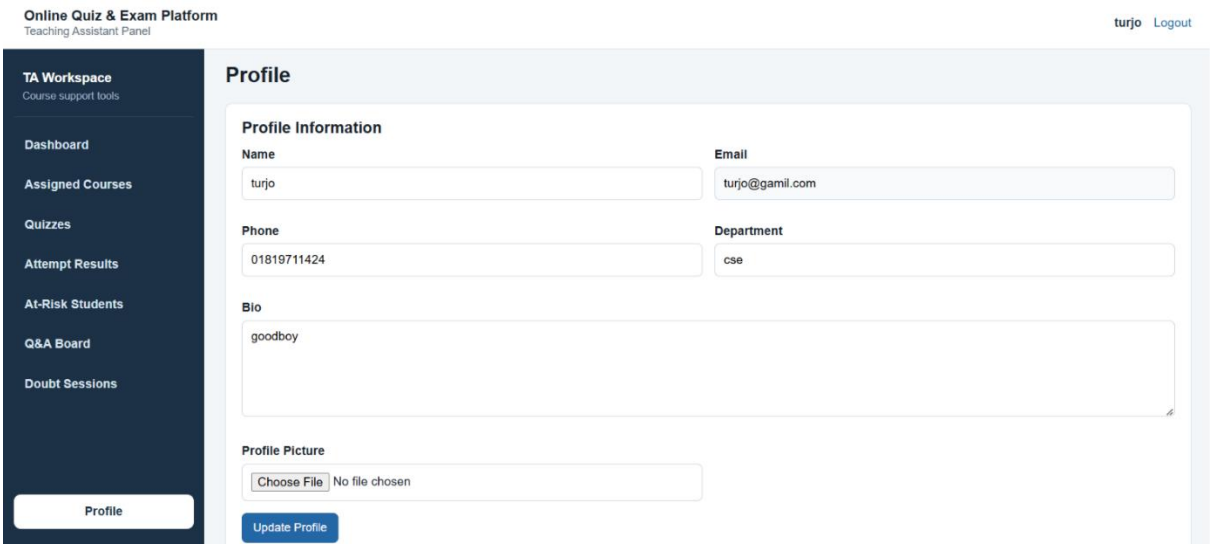
    $_SESSION['user_id'] = (int)$user['id'];
    $_SESSION['name'] = $user['name'];
    $_SESSION['email'] = $user['email'];
    $_SESSION['role'] = $user['role'];

    authRedirectByRole($user['role']);
}
```

Code Explanation

- authHandleLogin() reads the submitted email and password.
- userFindByEmail() loads the matching user record from the database.
- The active-status check blocks inactive accounts from logging in.
- password_verify() compares the submitted password with the stored password hash.
- Session values store the logged-in user's id, name, email, and role.
- authRedirectByRole() sends a TA to `ta.php?action=dashboard`.
- taProfile() loads the current TA profile and renders `views/ta/profile.php`.
- taUpdateProfile() updates only users whose role is `ta`, which protects other account types from this TA profile update.

Screenshot



Feature: View Assigned Courses

Description

Purpose: TAs can see all courses they are assigned to support.

User interaction: The TA clicks Assigned Courses in the sidebar and sees course title, instructor, subject, status, enrolled count, and quick action links.

Why needed: TAs should only work inside courses officially assigned to them

File: controllers/TaController.php

```
function taAssignedCourses($conn)
{
    $pageTitle = 'Assigned Courses';
    $courses = taGetAssignedCourses($conn, taCurrentId());
    taView('assigned_courses', compact('pageTitle', 'courses'));
}
```

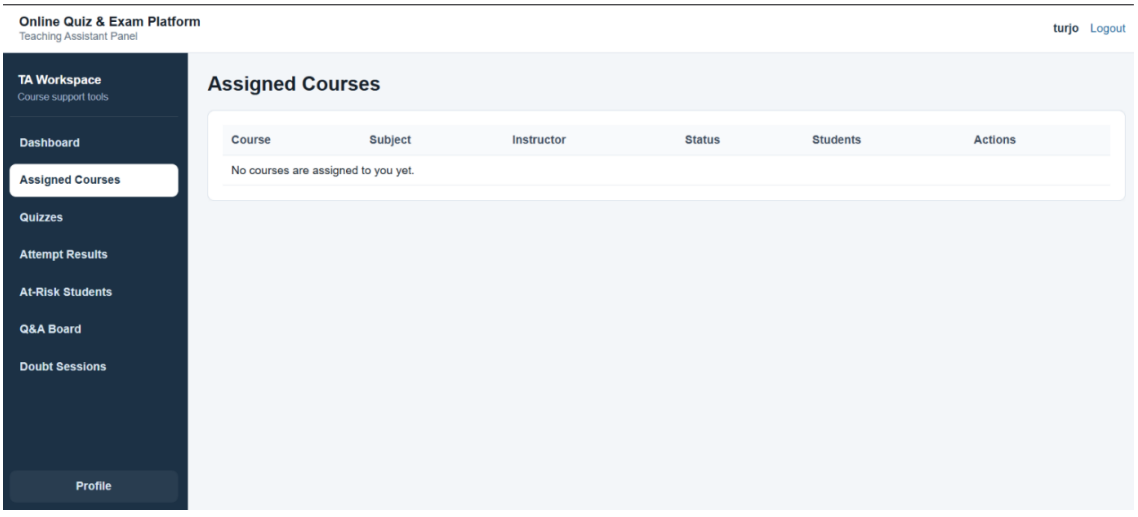
`models/TaCourseModel.php` - PHP

```php
function taGetAssignedCourses($conn, $taId)
{
    $sql = "SELECT courses.*, subjects.name AS subject_name,
                   users.name AS instructor_name,
                   COUNT(enrollments.id) AS enrolled_count
            FROM course_tas
            INNER JOIN courses ON courses.id = course_tas.course_id
            INNER JOIN subjects ON subjects.id = courses.subject_id
            INNER JOIN users ON users.id = courses.instructor_id
            LEFT JOIN enrollments ON enrollments.course_id = courses.id
            WHERE course_tas.ta_id = ?
            GROUP BY courses.id, subjects.name, users.name
            ORDER BY courses.created_at DESC";
    return dbSelectAll($conn, $sql, 'i', array((int)$taId));
}
```

Code Explanation

- taAssignedCourses() gets the current TA id from the session.
- taGetAssignedCourses() starts from `course_tas`, the assignment table.
- The query joins `coursessubjects`, and instructor `users` to show readable course details.
- LEFT JOIN enrollments counts enrolled students for each course.
- `WHERE course_tas.ta_id = ?` restricts the result to the logged-in TA only.
- The controller sends the result to the assigned courses view.

Screenshot



Feature: View Course Details, Students, and Quizzes

Description

Purpose: A TA can open an assigned course and see its core information, enrolled students, quizzes, materials, announcements, Q&A, and doubt sessions.

User interaction: The TA opens a course detail page from Assigned Courses.

Why needed: Course-level support work needs one place where the TA can inspect all important course data.

.**File:** controllers/TaController.php

```
function taCourseDetail($conn)
{
    $taId = taCurrentId();
    $courseId = taInt('id');
    taRequireCourseOrBack($conn, $courseId, $taId);

    $course = taGetAssignedCourse($conn, $courseId, $taId);
    $stats = taGetCourseQuickStats($conn, $courseId, $taId);
    $students = taGetCourseStudents($conn, $courseId);
    $quizzes = taGetQuizzes($conn, $taId, $courseId, '', '');

    taView('course_detail', compact('course', 'stats', 'students', 'quizzes'));
}
```

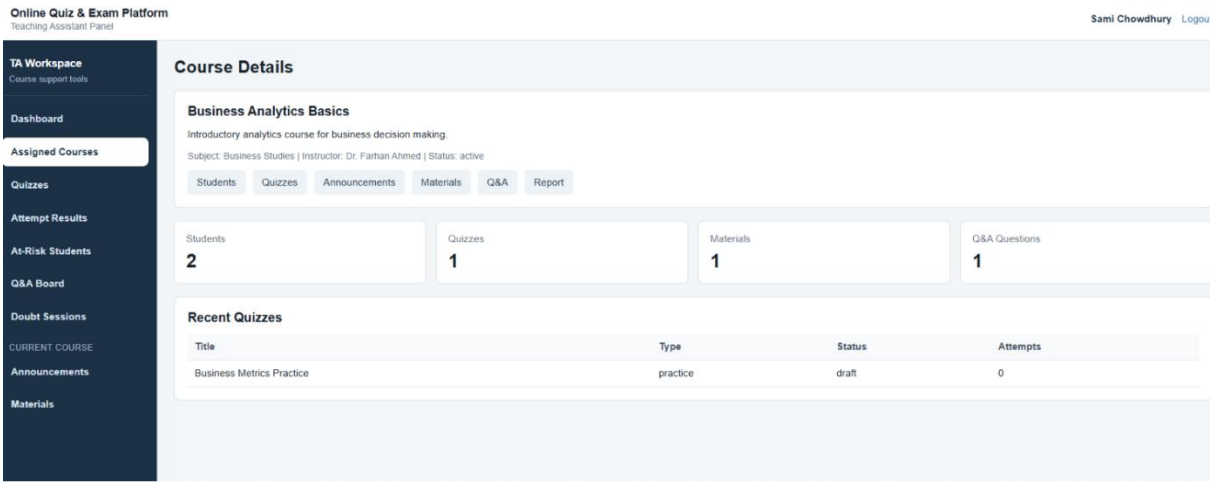
`config/ta_helpers.php` - PHP

```php
function taCanAccessCourse($conn, $courseId, $taId)
{
    $sql = "SELECT id FROM course_tas WHERE course_id = ? AND ta_id = ? LIMIT 1";
    return dbSelectOne($conn, $sql, 'ii', array((int)$courseId, (int)$taId)) !== null;
}
```

Code Explanation

- `taCourseDetail()` reads the course id from the URL.
- `taRequireCourseOrBack()` checks that the course belongs to the logged-in TA.
- `taGetAssignedCourse()` loads course information.
- `taGetCourseQuickStats()` loads summary counts for the course.
- `taGetCourseStudents()` loads enrolled students.
- `taGetQuizzes()` loads all quizzes for the selected course.
- `taCanAccessCourse()` uses the `course_tas` table to prevent access to unassigned courses.

Screenshot



Feature: View Assigned Courses

Description

Purpose: TAs can see all courses they are assigned to support.

User interaction: The TA clicks Assigned Courses in the sidebar and sees course title, instructor, subject, status, enrolled count, and quick action links.

Why needed: TAs should only work inside courses officially assigned to them

.**File:** controllers/TaController.php

```
function taAssignedCourses($conn)
{
    $pageTitle = 'Assigned Courses';
    $courses = taGetAssignedCourses($conn, taCurrentId());
    taView('assigned_courses', compact('pageTitle', 'courses'));
}
```

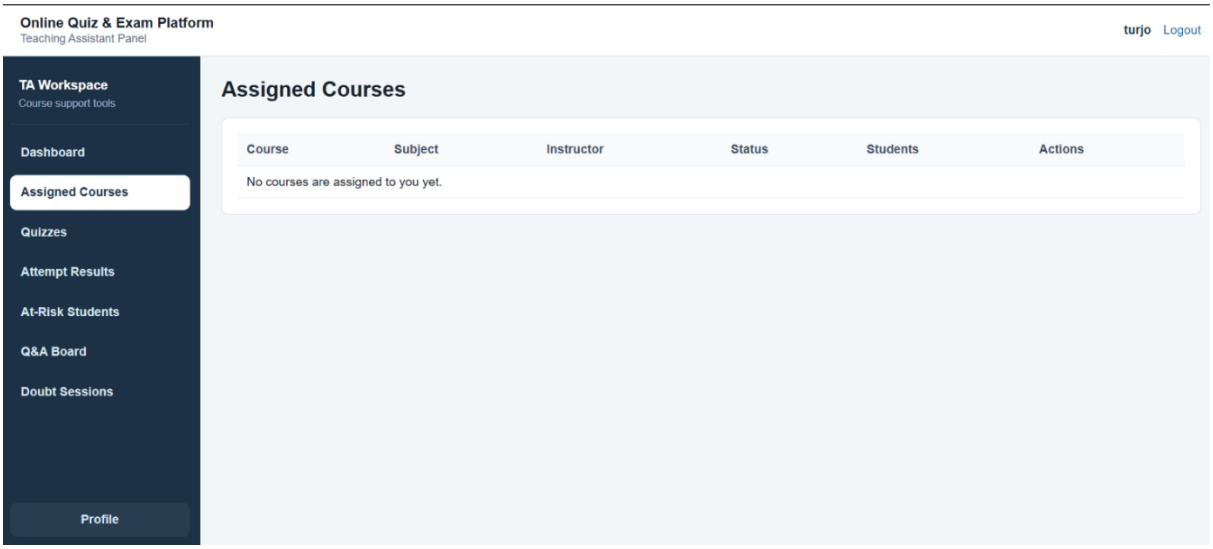
`models/TaCourseModel.php` - PHP

```php
function taGetAssignedCourses($conn, $taId)
{
    $sql = "SELECT courses.*, subjects.name AS subject_name,
                   users.name AS instructor_name,
                   COUNT(enrollments.id) AS enrolled_count
            FROM course_tas
            INNER JOIN courses ON courses.id = course_tas.course_id
            INNER JOIN subjects ON subjects.id = courses.subject_id
            INNER JOIN users ON users.id = courses.instructor_id
            LEFT JOIN enrollments ON enrollments.course_id = courses.id
            WHERE course_tas.ta_id = ?
            GROUP BY courses.id, subjects.name, users.name
            ORDER BY courses.created_at DESC";
    return dbSelectAll($conn, $sql, 'i', array((int)$taId));
}
```

Code Explanation

- taAssignedCourses() gets the current TA id from the session.
- taGetAssignedCourses() starts from `course_tas`, the assignment table.
- The query joins `coursessubjects`, and instructor `users` to show readable course details.
- LEFT JOIN enrollments counts enrolled students for each course.
- `WHERE course_tas.ta_id = ?` restricts the result to the logged-in TA only.
- The controller sends the result to the assigned courses view.

Screenshot



Feature: Create Practice Quizzes

Description

Purpose: TAs can create practice quizzes for assigned courses.

Userinteraction: The TA chooses Create Practice Quiz, fills title, description, marks, pass mark, time limit, and availability dates.

Why needed: Practice quizzes help students prepare before graded exams.

File: controllers/TaController.php

```php
case 'create_practice_quiz':
    taPracticeQuizForm($conn, 0);
    break;

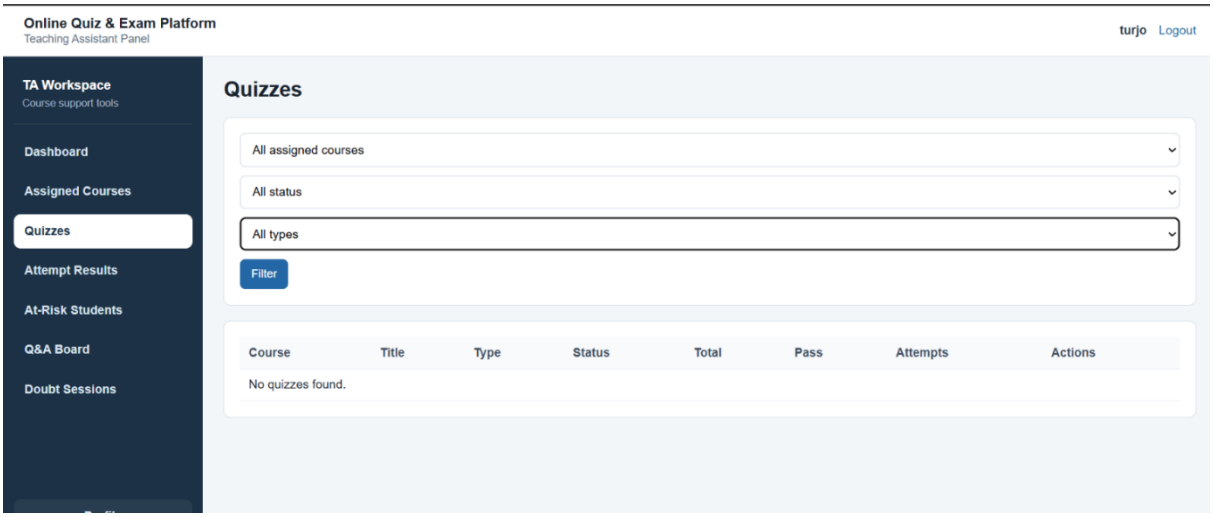
function taPracticeQuizForm($conn, $quizId)
{
    $taId = taCurrentId();
    $courseId = taInt('course_id');
    taRequireCourseOrBack($conn, $courseId, $taId);

    if (taIsPost()) {
        $old['title'] = isset($_POST['title']) ? trim($_POST['title']) : '';
        $old['created_by'] = $taId;
        $quizId = taCreatePracticeQuiz($conn, $old);
        taRedirect('ta.php?action=questions&quiz_id=' . (int)$quizId);
    }
}
```

Code Explanation

- The route action `create_practice_quiz` opens the quiz form.
- `taPracticeQuizForm()` verifies that the TA can access the course.
- On POST, form values are collected and validated.
- The logged-in TA id is stored in `created_by`.
- `taCreatePracticeQuiz()` inserts the quiz into the `quizzes` table.
- The query hardcodes `quiz_type = 'practice'`, ensuring the quiz is a practice quiz.
- The query hardcodes `status = 'draft'`, meaning it does not immediately publish to students.

Screenshot



Feature: Manage Question Bank and Options

Description

Purpose: TAs can add, edit, delete, and copy questions for quizzes in assigned courses.

User interaction: From a quiz, the TA opens Questions, adds question text, marks, four options, and one correct answer.

Why needed: Practice quizzes need reusable questions and answer options.

.**File:** controllers/TaController.php

```
function taQuestionForm($conn, $questionId)
{
    $taId = taCurrentId();
    $quizId = taInt('quiz_id');

    if ($quizId <= 0 || !taCanAccessQuiz($conn, $quizId, $taId)) {
        taRedirect('ta.php?action=quizzes');
    }

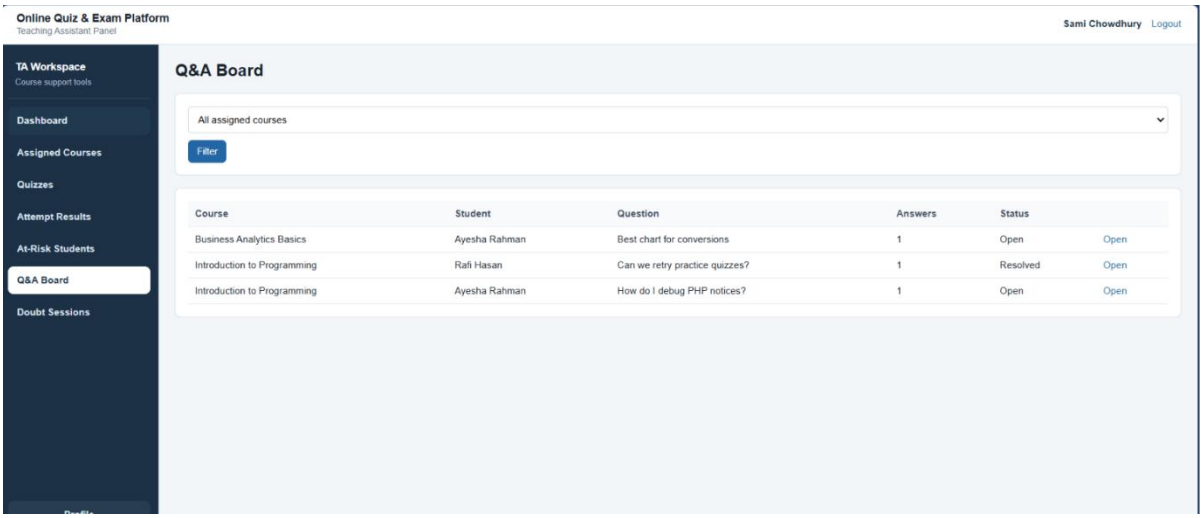
    if (taIsPost()) {
        $old['question_text'] = trim($_POST['question_text']);
        $old['marks'] = trim($_POST['marks']);
        $preparedOptions = taPrepareOptions($_POST['options'], $_POST['correct_option'], $errors);

        $questionId = taCreateQuestion($conn, $quizId, $taId, $old);
        taReplaceOptions($conn, $questionId, $preparedOptions);
    }
}
```

Code Explanation

- taQuestionForm() checks that the selected quiz belongs to one of the TA's assigned courses.
- The submitted question text and marks are read from POST data.
- taPrepareOptions() validates the four answer options and correct option.
- `taCreateQuestion()` inserts the question with quiz id, marks, order, and creator id.
- taReplaceOptions()` first removes old options for the question.

Screenshot



Feature: View Student Attempt Results

Description

Purpose: TAs can view quiz attempt results for students in assigned courses.

User interaction: The TA opens Attempt Results and filters by course or quiz.

Why needed: TAs need scores, attempt timestamps, and pass/fail status to monitor student progress.

.**File:** controllers/TaController.php

```
function taAttemptResults($conn)
{
    $taId = taCurrentId();
    $courseId = taInt('course_id');
    $quizId = taInt('quiz_id');

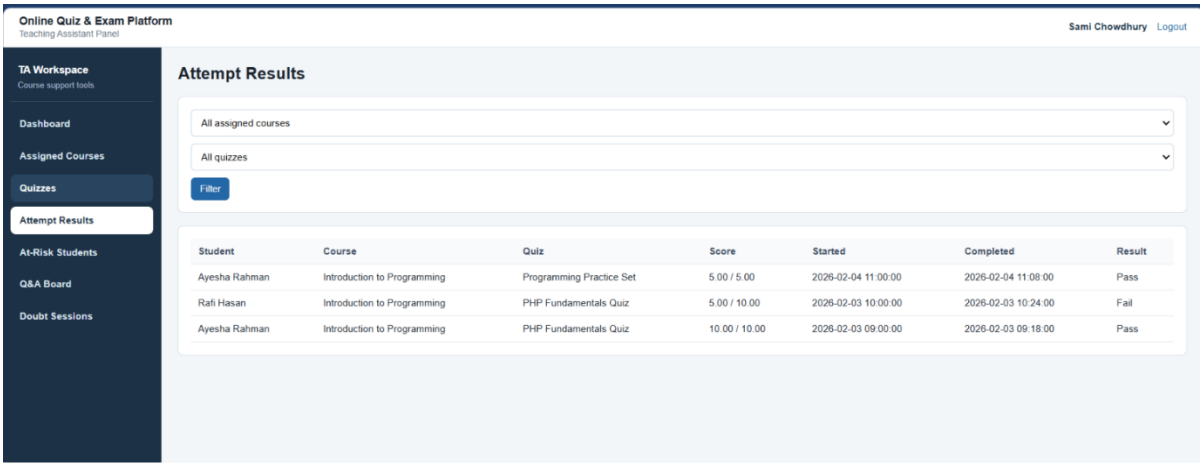
    $courses = taGetAssignedCourses($conn, $taId);
    $quizzes = taGetQuizzes($conn, $taId, $courseId, '', '');
    $attempts = taGetAttemptResults($conn, $taId, $courseId, $quizId);

    taView('attempt_results', compact('courses', 'quizzes', 'attempts', 'courseId', 'quizId'));
}
```

Code Explanation

- taAttemptResults()reads optional filter values from the URL.
- It loads assigned courses and quizzes for filter dropdowns.
- taGetAttemptResults() loads attempts only for the TA's assigned courses.
- The view prints score, total marks, start time, completion time, and calculated Pass/Fail text.

Screenshot



Feature: Identify At-Risk Students

Description

Purpose: TAs can view quiz attempt results for students in assigned courses.

User interaction: The TA opens Attempt Results and filters by course or quiz.

Why needed: TAs need scores, attempt timestamps, and pass/fail status to monitor student progress.

.**File:** controllers/TaController.php

```
function taAttemptResults($conn)
{
    $taId = taCurrentId();
    $courseId = taInt('course_id');
    $quizId = taInt('quiz_id');

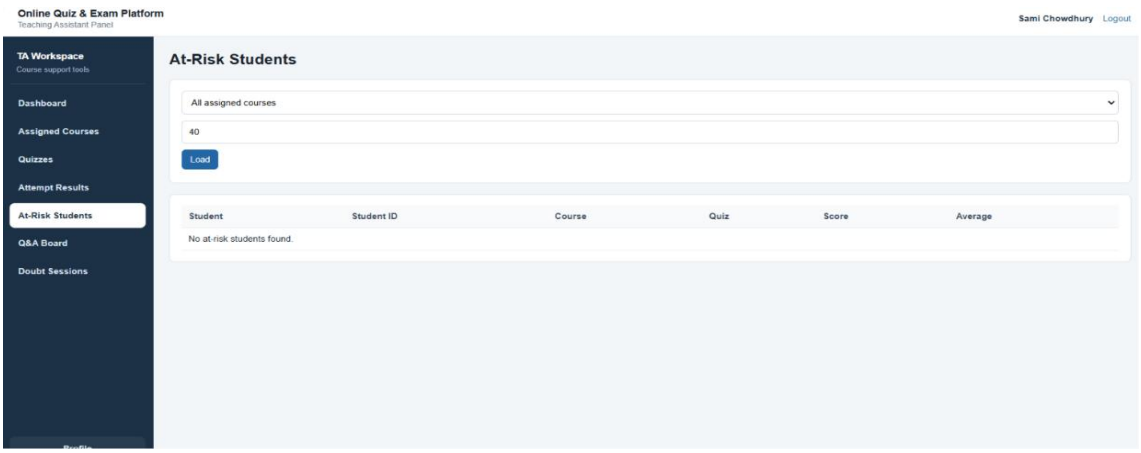
    $courses = taGetAssignedCourses($conn, $taId);
    $quizzes = taGetQuizzes($conn, $taId, $courseId, '', '');
    $attempts = taGetAttemptResults($conn, $taId, $courseId, $quizId);

    taView('attempt_results', compact('courses', 'quizzes', 'attempts', 'courseId', 'quizId'));
}
```

Code Explanation

- `taAtRiskStudents()` reads course id and threshold from the request.
- If no threshold is entered, it uses `taGetAtRiskThreshold()` from platform settings.
- The model query joins attempts with quizzes, courses, students, and TA assignments.
- The percentage calculation converts raw score into percentage.
- The final condition returns only scores below the threshold.

Screenshot



Feature: Monitor and Respond to Course Q&A

Description

Purpose: TAs can view course questions and post answers.

User interaction: The TA opens Q&A Board, selects a question, and submits an answer.

Why needed: Students need a course support channel for questions outside class.

.**File:** controllers/TaController.php

```php
function taQaDetail($conn)
{
    $taId = taCurrentId();
    $questionId = taInt('id');
    $question = taGetQaQuestion($conn, $questionId, $taId);

    if (taIsPost()) {
        $body = isset($_POST['body']) ? trim($_POST['body']) : '';
        taCreateQaAnswer($conn, $questionId, $taId, $body);
        taRedirect('ta.php?action=qa_detail&id=' . (int)$questionId);
    }
}
```

Code Explanation

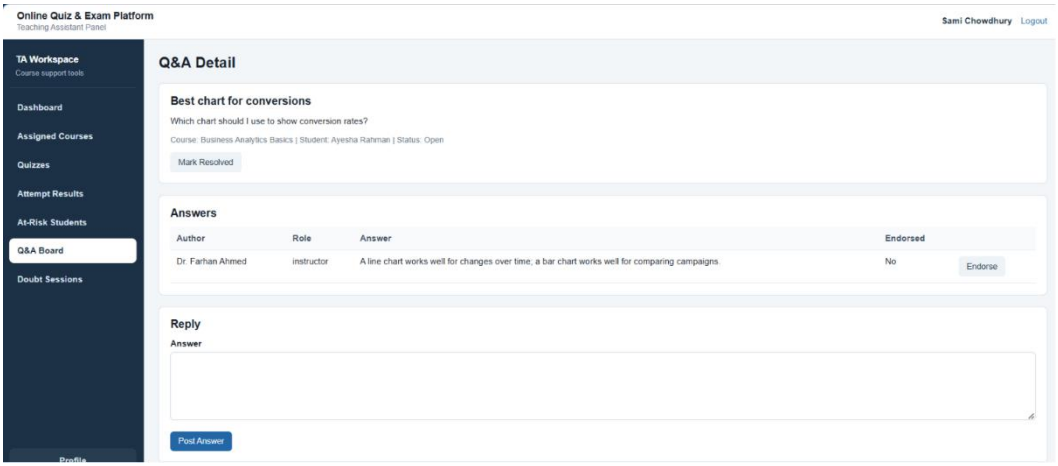
- `taQaDetail()` opens one Q&A question.

- taGetQaQuestion() ensures the question belongs to a course assigned to the TA.

- On POST, the answer body is read from the form.

- `taCreateQaAnswer()` stores the answer with question id and TA author id.Screenshot

Screenshot:



Feature: Course Summary Report

Description

Purpose: TAs can view a summary report for each assigned course.

User interaction: From a course, the TA opens Summary Report and sees total students, quiz counts, attempts, average score, at-risk count, Q&A count, and doubt session booking count.

Why needed: TAs need a quick course health overview.

File: `controllers/TaController.php`

```
function taCourseSummaryReport($conn)
{
    $taId = taCurrentId();
    $courseId = taInt('course_id');
    taRequireCourseOrBack($conn, $courseId, $taId);

    $threshold = taGetAtRiskThreshold($conn);
    $course = taGetAssignedCourse($conn, $courseId, $taId);
    $report = taGetCourseSummaryReport($conn, $courseId, $taId, $threshold);
    taView('course_summary_report', compact('course', 'report', 'threshold'));
}
```

File: models/TaReportModel.php

```
function taGetCourseSummaryReport($conn, $courseId, $taId, $threshold)
{
    return array(
        'total_students' => dbSelectOne($conn, "SELECT COUNT(*) AS total FROM enrollments WHERE course_id = ? AND status = 'active'", 'i', array((int)
        $courseId)),
        'attempts' => dbSelectOne($conn, "SELECT COUNT(attempts.id) AS total, AVG(CASE WHEN quizzes.total_marks > 0 THEN (attempts.score / quizzes.total_marks)
        * 100 ELSE attempts.score END) AS average_score FROM attempts INNER JOIN quizzes ON quizzes.id = attempts.quiz_id WHERE quizzes.course_id = ?", 'i',
        array((int)$courseId)),
        'at_risk' => dbSelectOne($conn, "SELECT COUNT(DISTINCT attempts.student_id) AS total FROM attempts INNER JOIN quizzes ON quizzes.id = attempts.quiz_id
        WHERE quizzes.course_id = ? AND (CASE WHEN quizzes.total_marks > 0 THEN (attempts.score / quizzes.total_marks) * 100 ELSE attempts.score END) < ?",
        'id', array((int)$courseId, (float)$threshold))
    );
}
```

Code Explanation

- The controller verifies that the TA is assigned to the selected course.

- The threshold is loaded from platform settings.

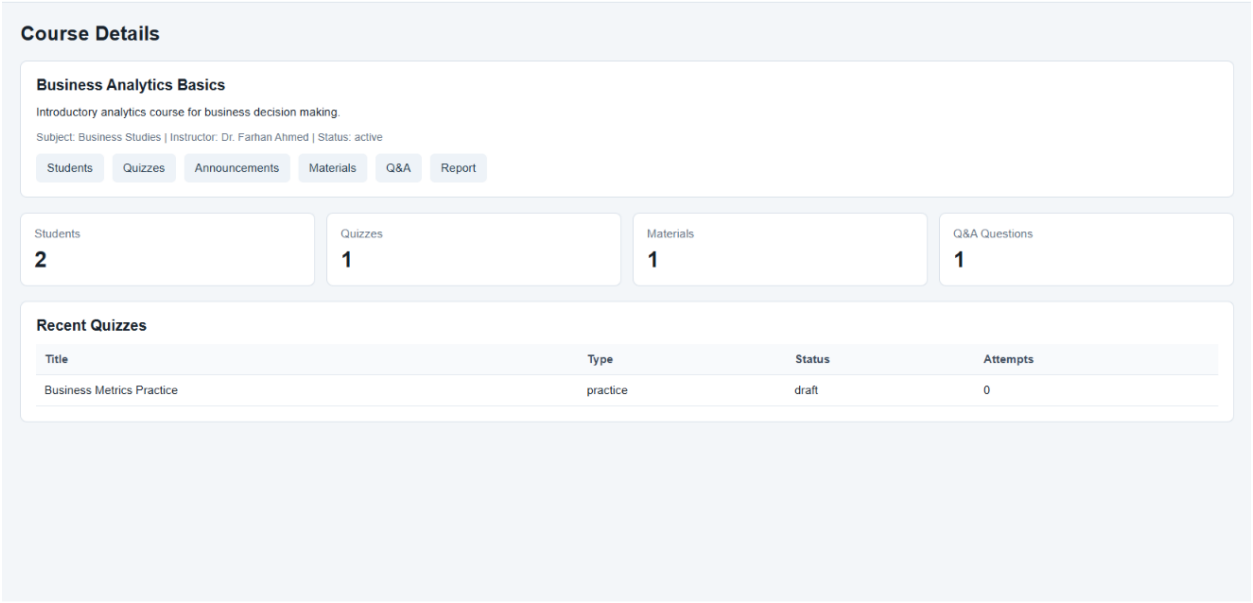
- The report model returns an array of count and average queries.

- `total_students` counts active enrollments.

- `attempts` counts attempts and calculates average score percentage.

- `at_risk` counts distinct students below the threshold.

Screenshot:



Chapter 4: Conclusion & Future Work

4.1 Summary

The Admin and Teaching Assistant modules of the Online Quiz & Exam Platform were developed as part of a multi-tier web application using PHP, MySQL, HTML, CSS, JavaScript, and the MVC pattern. The Admin module was built to manage platform-level operations such as user management, instructor approval, Teaching Assistant account creation, subject management, course and quiz monitoring, platform announcements, reports, settings, integrity reports, and audit logs. The Teaching Assistant module was built to support assigned courses by managing practice quizzes, questions, student attempt results, at-risk students, course materials, Q&A discussions, announcements, doubt sessions, and course summary reports.

The main goal of the project was to create secure and organized Admin and Teaching Assistant role-based features for the Online Quiz & Exam Platform. This goal was achieved by using PHP sessions for authentication, role-based access control for protected pages, MySQL database integration, mysqli prepared statements for secure queries, and separate models, views, and controllers to maintain the MVC structure. AJAX features were also implemented using XMLHttpRequest and JSON, including live user search for the Admin module and at-risk student filtering for the Teaching Assistant module.

During development, some challenges were faced, such as maintaining the MVC structure across many files, handling shared login and logout for multiple roles, writing database queries involving related tables, and implementing AJAX features correctly. These challenges were solved through proper file organization, repeated testing, server-side validation, session checking, and debugging.

4.2 Challenges Faced

During the development of the project, several practical challenges were faced. One major challenge was creating a shared login system for all roles. The system needed one common login page for Admin, Student, Instructor, and Teaching Assistant users, but each user had to be redirected to the correct dashboard based on their role. At first, managing this flow was confusing because the login system had to work for every role, not only one specific module. This was solved by checking the user's role from the database after successful authentication and then redirecting the user to the proper role-based page.

Another challenge was maintaining proper role-based access control. During testing, some protected pages could be accessed directly through the URL, so extra session and role checking had to be added. This was solved by checking the logged-in user's session and role before loading any protected Admin or Teaching Assistant page.

Maintaining the database properly was also a challenge because the project used many connected tables such as users, courses, course_tas, enrollments, quizzes, attempts, announcements, Q&A, materials, doubt sessions, reports, and audit logs. It was important to keep table relationships correct and avoid inserting inconsistent data. This was solved by carefully following the shared database schema, using proper foreign key relationships, testing sample data, and using mysqli prepared statements for secure database operations.

GitHub merging was another challenge because different group members worked on different roles of the same project. There was a possibility of file conflicts and duplicate files during merging. This was handled by keeping role-based files organized in separate controllers, models, views, CSS, JavaScript, and API files, and by checking changes carefully before merging them into the main project.

Overall, these challenges were solved through step-by-step testing, debugging, proper session checking, careful database management, organized MVC file structure, and controlled GitHub collaboration.

4.3 Future Improvements

The Admin and Teaching Assistant modules can be improved with the following features:

• Add advanced analytics and visual reports for Admin, such as charts for user activity, course performance, quiz attempts, pass rates, and subject-wise performance.

• Add a real-time notification system for important updates, such as instructor approval, platform announcements, integrity reports, at-risk student alerts, and doubt session changes.

• Improve the at-risk student tracking system so that Teaching Assistants can flag students, add follow-up notes, and monitor their improvement over time.

• Add automatic email or dashboard reminders for scheduled doubt sessions, pending reports, and important administrative actions.

• Improve the user interface to make the Admin and Teaching Assistant dashboards more responsive and easier to use on mobile and tablet devices.

References

PHP Documentation Team. (2026). PHP Manual. PHP.net.

Oracle Corporation. (2026). MySQL Reference Manual. MySQL Official Documentation.

Apache Friends. (2026). XAMPP Control Panel and Local Server Environment.

GitHub Inc. (2026). GitHub Documentation and Version Control Guide.